

MBR Intelligent Site Assistant

User Guide · Version 0.9.8

A self-hosted conversational site search for WordPress.

No external APIs. No monthly fees. Nothing leaves your server on the visitor path.

No schema change. Version 0.9.8 completes the visibility work of 0.9.7: a PDF is no longer treated as published on the strength of a link from a page an access-control plugin has restricted (§ 2.1). It also closes a gap in the chat panel that some themes opened between the input box and the credit line (§ 2.2). Nothing here needs a reindex, and an index built by any earlier version is safe to carry forward — every result is re-checked against the live post before it is returned (§ 2.4), and rows the corrected rules exclude are removed in the background (§ 2.6). If you are coming from 0.9.6 or earlier, read § 2.11 and § 2.15: contents-page detection and PDF link offsets are applied while indexing, so run a full reindex when convenient.

Published by Little Web Shack · littlewebshack.com

Author: Robert Palmer · GPL-2.0-or-later

Contents

- 1 Introduction
- 2 What's new in version 0.9.8
- 3 How it works
- 4 Installation
- 5 First-time setup
- 6 The diagnostic dashboard
- 7 The widget: placement and customisation
- 8 Intents — managing through the admin UI
- 9 Synonyms — managing through the admin UI
- 10 Appearance — themes and glass effect
- 11 Indexing behaviour
- 12 Command line (WP-CLI)
- 13 REST API
- 14 Privacy, data, and rate limiting
- 15 Troubleshooting
- 16 Uninstalling and keeping data
- 17 Technical reference
- 18 Support and version information

CHAPTER 1

Introduction

The MBR Intelligent Site Assistant is a WordPress plugin that adds a conversational chat widget to your site. Visitors type questions in plain English, and the assistant responds with short messages and links to the most relevant pages on your site.

The plugin runs entirely on your own server. There is no external API call for each query, no monthly subscription, and no visitor data sent to a third party. Ranking is done in pure PHP and MySQL using a well-understood information retrieval algorithm (BM25), layered with intent matching for common questions and synonym expansion for natural phrasing.

1.1 What it does

- Adds a floating chat bubble, or an inline widget via shortcode.
- Indexes your published posts and pages automatically when they are saved — and any custom post types you tick in the admin.
- Indexes the text inside your PDF files (optional), so their contents are searchable alongside posts and pages.
- Splits long documents into overlapping passage chunks, so a relevant passage deep inside a long PDF ranks on its own merits.
- Matches common questions (“contact”, “pricing”) to pre-written answers, with HTML support for links and basic formatting.
- Expands queries with synonyms so “WP” finds content tagged “WordPress”.
- Logs queries and thumbs-up / thumbs-down feedback so you can tune responses, with a configurable retention period.
- Provides full admin UIs for intents, synonyms, appearance, content sources, and privacy — no code editing required.
- Five colour-coordinated themes plus an optional glassmorphism effect.
- Ships WP-CLI commands for reindexing and status checks.

1.2 What it is not

Despite the word *assistant*, this plugin is not a large language model and does not generate text. It retrieves and surfaces content you have already written, and answers a configurable list of canned questions. If a visitor asks something you have not written about, the assistant will politely say it could not find anything, and invite them to rephrase.

This is a deliberate design choice. The plugin produces predictable, auditable responses with no hallucination risk — and it does it without sending your visitors' questions to an outside service.

PDF indexing follows the same principle: it reads the text that is already inside your PDFs. It does not perform OCR on scanned images, and it does not decrypt protected files.

1.3 What the assistant will show a visitor

Everything in the index is reachable by anyone who can load your site. The search endpoint is public by necessity — a visitor asking a question is usually not logged in — and it runs no capability check. The plugin

therefore indexes only content that is already public: published, non-password-protected posts of the types you enable, plus PDFs meeting the rule in § 11.5.

Worth knowing. If you are evaluating this plugin for a site that holds anything sensitive, § 11.1, § 11.5 and § 14 are the three sections to read. They set out exactly what enters the index and what is stored about your visitors.

CHAPTER 2

What's new in version 0.9.8

Version 0.9.8 is a follow-up to the review that produced 0.9.7, and closes the one place that release left open: the PDF subsystem was still deciding for itself what counted as public. It also fixes a cosmetic fault in the chat panel that has been possible on every version of this plugin. There is no schema change, no settings change, and no reindex — the corrected rule applies from the moment the files are in place, and the index is tidied in the background.

Worth reading first, if this plugin runs on a site with membership or access-controlled content: § 2.1 completes the work begun in § 2.3, and is the reason this release exists. If you do not index PDFs, § 2.2 is the only section here that affects you.

2.1 A link only counts if the page doing the linking can be read

0.9.7 added `mbr_isa_can_index_post` so that a plugin which knows a post is restricted can say so, since nothing in the post row records it (§ 2.3). The filter runs on indexing, on save-time removal and on the search-time check alike, and it closed the case it was written for.

It did not reach the PDF subsystem, and that is what this release is about.

Whether a PDF counts as published is answered by asking what points at it (§ 11.5), and that question was put to the database: find a published, unprotected post, of a front-end viewable type, whose content or post meta mentions the file's path. Three of the four conditions in § 2.3 live in the post row and SQL can test all three. The fourth cannot be expressed as a WHERE clause at all — the filter is arbitrary PHP holding whatever the site's membership plugin knows, and no query can run it.

So a members-only page was correctly kept out of the index by the filter, and still counted as evidence that a PDF it links to had been published. The file's full text was indexed and returned to anonymous visitors, who could be told from the chat panel what the page itself would not show them. The gate 0.9.7 built, with a way round it one level down.

The scan now retrieves candidates rather than conclusions. Each is loaded and put through the same `is_publicly_readable()` that governs everything else, and the first that passes settles the question.

- **Both passes.** Post content and post meta are treated identically. The meta pass exists because page builders keep their content there (§ 2.25), which is also where a membership-protected page's PDF link is most likely to be — so fixing only the content pass would have fixed the rarer half.
- **The work is bounded.** Candidates are capped, and the batch is fetched in one query rather than one per candidate. Overrunning the cap fails closed: a file whose every examined referrer is restricted stays out of the index, which costs a search result rather than disclosing one. Raise the cap with `mbr_isa_pdf_reference_candidates` on a site where files are routinely linked from very many places.
- **Ordering is left to the database, deliberately.** An `ORDER BY` would force the whole `LIKE` scan to finish before the limit could apply, and that scan is the expensive half of the operation. Without one it stops as soon as the candidate list is full.
- **The index is tidied in the background.** Rows written under the old rule are removed by the same purge described in § 2.6, scheduled again on upgrade. Nothing stale can be served while it is pending, because § 2.4 re-tests every result against today's rules before it leaves the server — which is why this is

housekeeping rather than something you need to act on.

What this buys, and what it does not. There is now one definition of “public” in the plugin and four callers of it: post indexing, search-time revalidation, a PDF's parent, and a PDF's referrer. That is worth more than the particular leak it closes, because the next rule change lands in one function rather than four.

Worth knowing: the uploads directory. A PDF in `wp-content/uploads` is fetchable by anybody holding its URL, whatever WordPress thinks of the page that links to it, unless something on your server is protecting it. This release stops the plugin making a restricted document *searchable*. It cannot make the file itself private. A document that must not be read needs protecting at the server, or by a plugin that moves it out of the uploads path — the same as it always did.

2.2 The strip of background under the input box

With some themes, a band of a slightly different colour appeared between the input row and the credit line at the foot of the chat panel.

The panel is a column of four blocks — header, log, input row, credit line — and the last two share a background. A margin between them therefore does not read as spacing; it reads as a stripe of the panel showing through, because that is exactly what it is.

The margin was the host theme's. `<form>` is an element themes reliably style, a bottom margin on it is ordinary, and nothing in the widget's stylesheet said otherwise — so the gap appeared or did not appear entirely according to the theme, on every version of this plugin. It was never new, which is the answer to the obvious question.

- All four panel sections now zero their margins, and so do the input and the send button inside the form row.
- The rules are scoped through the panel rather than written as a single class. A single class beats a bare `form` selector but loses to `.entry-content form` — and that is the rule most likely to be responsible, since the shortcode embed renders inside the content wrapper the theme is styling. Same reasoning as the resting-state fix in 0.9.1, one level out.
- **Nothing to do beyond a cache clear.** The stylesheet is versioned by the plugin version, so 0.9.8 asks for a fresh copy — but a page cache or CDN holding the old asset will hide the fix. Clear both.

2.3 Published no longer means public on its own

Until now the plugin asked two questions before indexing a post: is it published, and is it free of a password. Both are necessary and neither is wrong. Together they are still narrower than the access control actually running on a real WordPress site.

A membership or paywall plugin routinely leaves a post with the status publish and no password while refusing the content to anonymous visitors. Nothing in the post row records that. The indexer reads the row, sees published and unprotected, and writes the body to an index served through an endpoint that performs no capability check — so a visitor could ask what the members-only page said and be told.

Two things change.

- **Post types are checked.** A post is indexed only if its type is viewable on the front end. A type registered with `'public' => false` has no permalink for a result to link to and no template to render it, and has no business being served through search whatever a stored setting says about it.

- **There is a new filter.** `mbr_isa_can_index_post` lets a plugin that does know about a restriction declare it. It runs on indexing, on save-time removal and on the search-time check in § 2.4 alike, so one filter covers every path.

```
add_filter( 'mbr_isa_can_index_post', function ( $allowed, $post ) {
    return my_plugin_is_members_only( $post->ID ) ? false : $allowed;
}, 10, 2 );
```

The filter is deny-only. It runs only after the built-in conditions have already passed, so returning true from it cannot publish a draft or unlock a protected post. It can tighten this gate; it cannot open it. A security boundary a third-party callback can widen is not a boundary.

The post-type test used is WordPress's own `is_post_type_viewable()`, rather than the obvious check of public and publicly queryable together. The obvious check is wrong: the built-in page type is public but deliberately not publicly queryable, because pages are resolved by path rather than by query variable. A naive test would have excluded every page on every site.

Why this does not live in the settings screen. Content Sources deliberately keeps a post type ticked while the plugin that registered it is deactivated, so that saving the form during a temporary deactivation cannot silently drop content from your configuration (§ 6.10). That is right for a settings list and a poor foundation for a security property: the stored value was written against whatever the type looked like then, and says nothing about what it looks like now. The indexer now decides for itself and the settings screen decides scope.

2.4 Every result is checked again before it is shown

The index is a cache of a decision taken at index time, and a cache can go stale. Every path that writes to the index re-checks visibility, and that was never the problem — the problem is that writing is not the only way content changes hands.

Four ways a row outlives the decision that created it:

- A membership or access-control plugin restricts a page without touching `post_status` or `post_password`, and without firing `save_post` in the way this plugin expects.
- A site upgrades from a version whose gates were narrower, and carries the older index until somebody clicks Reindex.
- A post type is re-registered as private.
- The indexing rules change — as they just did in § 2.3 — and a rule change cannot retract what the previous rules already wrote.

In all four the row is sitting in the table and the endpoint is public, so without a second look the answer is served.

From 0.9.7 there is a second look. Before a result leaves the server it is re-tested against the live post, and a row whose source has since become private, protected, restricted or deleted is discarded rather than returned.

Whatever is in the table, only content that passes today's rules today is shown.

This is the change to keep if you keep only one. It is not a fix for a particular leak so much as a floor beneath the whole class of them — which is also why an index built by any previous version of this plugin is safe to carry forward without a reindex.

- The number of rows examined per query is capped. A pathological case — an index full of stale rows, a quoted-phrase search with a large candidate pool — cannot turn one visitor question into hundreds of post lookups. Hitting the cap costs a shorter list of results, never a disclosure.
- Answers are memoised within a request, and for PDFs cached across requests where a persistent object cache is available. The expensive part of a PDF's visibility test is the scan for what links to it, and the cheap route — a published parent — is tried first, so most files never reach it.
- The Search Tester and the raw response payload report a `suppressed_stale` count. A non-zero figure means the index holds rows whose source is no longer publicly readable. Nothing leaked; they were dropped. It is a signal that the index wants tidying (§ 2.6).

One behaviour change worth knowing. Switching PDF indexing off now takes effect on search immediately. Previously the setting governed what entered the index, and files indexed while it was on stayed findable until a reindex. They are now suppressed at the point of asking, which is what unticking the box reads as meaning.

2.5 A link only counts if the page doing the linking is reachable

The recommended PDF rule indexes a file when something published points at it (§ 11.5). The test for “something published” asked for a post with the status publish, no password, and any post type other than a revision or an attachment.

That last clause is an exclusion list, and an exclusion list admits everything you did not think of. It accepted a link from a reusable block, a page-builder template, a private custom post type — anything with no front end at all. A PDF whose only reference lived somewhere unreachable therefore counted as published, and its full text became searchable.

It is the same mistake as gating on `post_status` alone, one level further out. A link is only evidence of publication if the thing doing the linking is itself reachable. The test now names the post types that qualify rather than the two that do not.

- All viewable post types are accepted, not the ones ticked under Content Sources. Whether a type is searchable is a scope question; whether its attachment is public is a visibility question, and 0.8.3 already separated those (§ 2.26). A PDF linked from a published product page is public whether or not product is a type you index.
- The post-meta pass is unchanged and still covers page builders that keep their content there.
- On most sites this changes nothing. It matters where PDFs are linked from reusable blocks, or from a custom post type that is not front-end viewable.

2.6 Cleaning up an index built under the older rules

The rules in § 2.3 and § 2.5 govern what enters the index. They cannot retract what earlier rules already wrote, and previous releases handled that by flagging Reindex required and asking you to act — which left the old index queryable until you did.

That gap is gone, because § 2.4 closes it: an index built under the old rules cannot serve content the new rules exclude, whether or not anybody has clicked anything. What remains is untidiness rather than exposure, and it is dealt with quietly.

- A background pass starts shortly after the upgrade and walks the index in small batches, removing rows whose source no longer qualifies. It runs on WP-Cron, in its own request, so nothing holds an admin page open.

- There is no admin notice and no reindex prompt, because there is nothing you need to do. A site whose WP-Cron is broken ends up with an untidy index, not a leaking one.
- A full reindex achieves the same thing and more, and cancels the need for the pass. Reach for the purge when you want the index corrected without paying to rebuild it.

To do it immediately, or to see what it would remove first:

```
$ wp mbr-isa purge --dry-run
$ wp mbr-isa purge
```

The dry run lists the affected posts by ID and title. It is also a straightforward way to find out what previous versions of the plugin were willing to index on your site.

2.7 Search no longer measures the whole index on every query

BM25 normalises each field against its own length and against the corpus average for that field (§ 11.4). Both figures were being derived, on every single visitor query, from an aggregate across the entire postings table.

That is affordable on a small site and an architectural problem on a large one. The cost grows with the size of the index rather than with the difficulty of the question, so the slowest queries on a big site were the ones that matched almost nothing.

Two changes, and neither alters a single score.

- Only documents that actually contain a query term are measured. The scorer already skipped any document with no recorded length, so a document matching nothing contributed nothing — the rest of the table was being loaded purely to be ignored.
- Corpus averages and the document count are calculated once and cached until an index write invalidates them. They are properties of the index, not of the question, and they change exactly when the index does.

Invalidation is by index generation rather than by clock. An expiry time would only ever be a guess at when the index last changed, and a wrong guess in either direction is either stale ranking or a pointless recalculation.

Nothing to retune. Scores, confidence levels and result ordering are identical to 0.9.6. If you have adjusted field weights or the BM25 constants, this release does not disturb them.

2.8 One rating per answer

A feedback token is valid for an hour and was not consumed by use, so anybody holding one could keep submitting ratings against the same answer for the rest of that window and move your statistics on their own. Rate limiting bounded how fast, not how many.

Feedback is now accepted once per query. A second submission returns 409, after the token check rather than before it — whether a given query has already been rated is information, and answering that for an unsigned request would hand an ID-walker the read primitive the token exists to deny them.

The widget already replaced the rating buttons with a confirmation on the first click, so this closes the gap between what the interface offers and what the endpoint accepts rather than taking anything away from a visitor.

A related irritation goes with it. If the first submission succeeded but its response never arrived — a dropped connection, a timeout — the retry used to report a failure for a rating that had in fact been recorded, and put the

buttons back for a third attempt. The widget now treats 409 as success, because that is what it is.

2.9 Update packages are checked against their manifest

The plugin updates itself from a manifest on GitHub pointing at a package on littlewebshack.com (§ 4.3). That is an ordinary arrangement for a privately distributed plugin, and it does mean the update path is a code-delivery path whose integrity rests on two separate hosts — with no verification of its own, since WordPress only checks packages that came from WordPress.org.

The manifest may now carry a SHA-256 of the package, and where it does, the downloaded file is checked against it before WordPress is allowed to unpack anything. A mismatch stops the update and reports both digests rather than installing and hoping.

- Backward compatible. A manifest with no checksum key updates exactly as it always has, and a malformed one is treated as absent rather than as a failure — a typo in a manifest should not be able to wedge updates for every install.
- Define `MBR_ISA_REQUIRE_SIGNED_UPDATES` as true in `wp-config.php` to refuse a package that cannot be verified. Opt-in, because verification you can forget to publish will eventually block a legitimate release at an inconvenient moment.
- The digest is bound to the download URL it was published alongside. A digest is a statement about one specific file, so applying a remembered one to a package fetched from somewhere else would be worse than not checking.

What this buys, stated honestly. It binds the package to the manifest, so tampering with the package host alone cannot ship code — the digest will not match. It does nothing against somebody who controls the manifest, since they would simply publish a digest of their own package. Raising that bar means signing the manifest itself with a key shipped in the plugin, which is a separate piece of work.

2.10 Also in 0.9.7, and two things deliberately left alone

A full reindex now asks the host for more execution time and memory before it starts. Both requests are advisory — a host with a hard PHP-FPM ceiling will refuse them — so WP-CLI remains the reliable route on a large site (§ 12.1).

The chat panel now collapses towards the floating bubble when it is closed, rather than fading where it stands. It is a stylesheet change with no setting attached, and the existing reduced-motion rules switch it off along with every other transition. The footer credit links to the plugin's own page rather than the site root.

Two recommendations from the same review were considered and not taken in this release.

Background indexing. The full reindex still runs to completion inside one request. Converting it to genuine background batches is a feature with its own interface, resume semantics and failure modes, and it does not belong in a release whose job is closing an access-control gap. It is the next substantial piece of work.

Query logging remains on by default. Logging free-text visitor input is worth being deliberate about, and § 14.2 says so plainly. But the plugin's entire tuning loop runs through the query log — the zero-result list, the feedback buttons, the Recent queries table — and a fresh install with all of that dark reads as broken rather than as careful. It is a choice you can make on the Privacy panel before the widget goes live, and the widget is off by default precisely so that you get the chance.

2.11 A contents page no longer hides its own chapter

Introduced in 0.9.6, which was never published, and shipping here.

Results collapse to one passage per document, so a document is represented in the results by its single best-matching chunk. Where that chunk was a contents page, the chapter it lists became unreachable: the document's one result was a run of section titles, and no amount of rephrasing would reach the prose behind them.

This was self-defeating for precisely the query it was meant to serve. Contents passages are demoted rather than excluded (§ 11.6) so that a visitor searching for a section title can still find it — but searching a chapter title landed on the contents-page line naming that chapter, rather than on the chapter.

Where a document has any matching passage that is not a contents page, that passage now represents it. A contents chunk is used only when nothing else in the same document matched — which is what the demotion was always trying to express, applied within a document rather than across the whole index.

- Nothing is lost. A search that matches only a contents page still returns it.
- The document's position in the results follows the score of whichever passage ends up representing it, so a substitution can move it down the list. That is correct: the score now reflects what the visitor will actually be shown.
- `contents_score_penalty` is unchanged and still governs ranking between documents (§ 17.1). The two work together: the penalty decides how a contents page competes with other documents, this decides whether it may speak for its own.
- Set `contents_prefer_body` to false for the previous behaviour.

And in 0.9.7, the detection behind it. None of the above can fire unless a passage is recognised as a contents page in the first place, and two things stopped that happening on documents where it matters most.

- **An entry did not have to carry a full stop.** The test looked for “3. How it works” and a great many documents write “3 How it works” instead — this guide among them. A contents page set that way registered no entries at all, so it was never demoted and never stood aside for the chapter it lists. Both forms are now recognised. The dotless one requires a capital after the number, since without the punctuation there is nothing else separating “7 The widget” from “30 requests per minute” in a data table.
- **The run is measured, not the whole passage.** The old test asked whether the entire chunk read as connective prose, which works when a contents page fills its passage and fails when it does not. A short contents list with a title page on one side and the opening of chapter one on the other is still a dense list of every topic in the document — exactly the thing that out-scores the chapter somebody asked for — but the prose around it lifted the average and the list was waved through. The span covered by the entries is now measured on its own, and has to account for a fifth of the passage before it counts.
- Decimals are still excluded, so a settings reference table full of values like 1.5 and 0.75 does not register as a contents page. That mattered more than catching every last one: a false positive costs a real passage its ranking.
- **And the overlap was handing the list to its neighbour.** Consecutive chunks share 50 words so that a sentence straddling a boundary survives whole in one of them (§ 11.6). A contents run is the one structure that inverts the benefit: its opening entries were copied into the chunk before it, which is otherwise ordinary prose — so that chunk was not a contents page by any reasonable test, was not demoted, and was free to represent the document. The correctly-flagged chunk sat demoted beside it while the copy won. A contents run now takes a chunk of its own, with no overlap across its edges, so the prose either side carries only prose.

- **And the replacement had to be in the candidate pool.** The substitution above could only reach for a chunk already among the top scorers across the whole index. That is ample when a query term is distinctive — search “Introduction” and little else competes, so the chapter is right there. It fails exactly where a contents page is most tempting: search words that appear in every document on the site and the pool fills with short dense passages, the contents chunk among them, while the chapter that should have replaced it ranks sixtieth and is never looked at. The substitution now considers every chunk that matched, not only the ones in the pool, and queries only for documents actually facing the problem — in practice none or one.

This one wants a reindex. Whether a passage is a contents page is decided while indexing and stored on the row, so documents already in the index keep the answer they were given under the old test until they are rebuilt. Nothing else in 0.9.7 needs it, and nothing breaks in the meantime — run `wp mbr -isa reindex` when it suits you.

2.12 Multi-word searches open on the right occurrence

A snippet opened at the first occurrence of *any* query term rather than where the terms appeared together. Search a long passage for “privacy, data, and rate limiting” and, if the word “rate” happened to appear in its opening line, the snippet opened there — leaving the phrase you actually wanted further down, in the next snippet.

Each occurrence is now scored by how many distinct query terms fall within a window around it, and the densest wins. Ties go to prose over code, then to the earliest position, so a single-term query behaves exactly as it always did.

A useful side effect. The snippet builder and the deep-link builder share this helper, precisely so the text shown and the place linked to can never disagree. Sharpening it therefore also sharpens which page a PDF result opens at, and where on that page.

2.13 Sentence breaks in PDF snippets

PDFs carry no block structure, so their snippets arrived as one unbroken run. Sentence boundaries now stand in for paragraphs. It is artificial — the document has no paragraph there, only a full stop — but a three-sentence snippet is markedly easier to scan broken up than as a single line.

The guards matter more than the rule, because technical prose is full of full stops that end nothing. Left alone: section references such as § 11.5., version numbers, initials as in “R. Palmer”, and abbreviations including e.g., i.e., etc., Fig. and Ltd. A very short fragment stays attached to the sentence before it rather than stranded on a line of its own.

Posts and pages are unaffected — they have real block structure and already break on it (§ 2.14).

2.14 Snippets show paragraph breaks

Block boundaries in post content were flattened to spaces, so a snippet spanning two paragraphs ran them together as one sentence. They now render as line breaks, which makes a multi-paragraph snippet noticeably easier to read.

- Applies to posts and pages, which carry real block structure from their HTML.

- PDF snippets are unchanged. A PDF's line breaks are hard wraps from the original layout — a break every dozen words, mid sentence — so reproducing them would make snippets more ragged, not more readable. They were also collapsed before the passage was stored, so they are not available to reinstate.
- A page boundary inside a PDF passage stays a space: a page break is a layout event, not a change of thought, and breaking there would imply a paragraph that is not present.
- `<br>` joins `<mark>` as the only markup the responder introduces, and is added after escaping so it cannot come from the source text.

2.15 PDF links aim at the passage, not the top of the page

A PDF result has always linked with `#page=N`, which scrolls the top of that page into view. When the matching passage is halfway down the page the visitor lands a screen above it and has to hunt — on a dense page, most of a page of scrolling.

Links now carry a vertical target as well, so the view starts near the passage rather than at the page's top edge.

How the position is worked out, and why it is an estimate.

Honestly: by guessing, carefully. The extractor reads the operators that move the text cursor but discards their operands, so nothing in the index knows the coordinates of any word. What it does know is how far through a page's words a passage begins and — new in 0.9.2 — how tall that page is, read from its MediaBox. Those two give a position, on the assumption that text is spread evenly down the page.

That assumption is wrong on any page with a figure, a table, or a short closing paragraph. The estimate is therefore biased upward by about a fifth of a page, so the passage lands at or below the top of the view rather than above it. The visitor scrolls down a little — the same direction as before, far less far. Overshooting the other way would push the passage off-screen above them, which is worse than the behaviour this replaces.

- A passage already near the top of its page gets no offset, so the link stays short and the reader's zoom is left alone.
- **Corrected in 0.9.4.** The first implementation scrolled with `view=FitH`, which also fits the page to the window width — opening documents at around 163% on a normal desktop instead of 100%. Links now use `zoom=scale, left, top`, which names the scale explicitly. It defaults to 100 and is configurable through `pdf_link_zoom`.
- Where the match falls on a later page than the chunk began on, the text there starts at the top of that page, so no offset is used.
- Pages rotated 90 or 270 degrees are measured on their rotated height, and a MediaBox inherited from the page tree is followed. If no height can be resolved for every page, no offsets are stored and links behave exactly as before.
- Any scrolling open parameter must also set a view, so a zoom level has to be named — there is no way to say “scroll here and leave the zoom alone”. Set `pdf_link_position` to false to switch positioning off entirely (§ 17.1).
- The two parameters use opposite origins: `view=FitH` measures up from the bottom-left in PDF user space, `zoom` measures down from the top-left. The stored offset follows the latter, which is why 0.9.4 needed its own reindex — an index built by 0.9.2 holds the mirror image.

Reindex required — from before 0.9.4 only. The offset is stored per passage, so it is written during indexing. Schema version 5 adds the column with a default of zero, and zero means “no offset”. If you are coming from 0.9.4 or later there is nothing to do; from earlier, every existing row keeps the old page-top behaviour until a full reindex rebuilds it. Nothing breaks in the meantime; the improvement simply does not appear.

2.16 Quoted phrase search

Wrapping a query in quotes searches for that exact run of words. "opening hours" returns only pages where those two words appear together, in that order, rather than pages about opening and pages about hours.

Straight and curly quotes both work, single and double — a phone keyboard produces curly ones without asking.

A quoted query also skips intent matching. That is the point of the feature as much as the phrase matching is: previously a query sharing a single word with an intent trigger got the canned answer and no search at all, so there was no way to ask for the words themselves. Synonym expansion is skipped too, since swapping in different words is the opposite of what an exact search requests.

How it works, and what it cannot do.

The index is a bag of words with no record of position, so a phrase cannot be resolved from the postings alone. Instead the stems find candidate passages cheaply, and each candidate's stored passage text — already held for snippet generation — is checked for the literal phrase. The check runs before chunks are collapsed to one row per document, so a phrase sitting in a document's third chunk is still found when its first chunk scores higher.

- Case is ignored, and any run of whitespace matches any other, so a phrase still matches across a line break in a PDF or a run of spaces in justified text.
- Titles are searched as well as passage text, so quoting a page title finds that page.
- Curly quotes and non-breaking spaces inside the phrase are folded to their plain equivalents before comparison, so a phrase pasted out of a word processor still matches.
- A single word in quotes is valid and meaningful — it demands that literal word, with no stemming leniency from synonyms and no intent interception.

The one real limitation. A phrase is only found if it survives whole inside one passage chunk. The 50-word overlap between consecutive chunks (§ 11.6) means a phrase straddling a boundary is normally present intact in one of them, but a very long quoted phrase can still fall between two. When nothing matches, the reply names the phrase and suggests searching without the quotes rather than implying the words are absent from the site.

2.17 Results alongside an intent answer

An intent used to end the pipeline: canned answer returned, search skipped entirely. That is right when the canned answer is the whole story, and wasteful when it is not — someone asking about opening hours may also want the contact page.

Where a query matches an intent and produces good search hits, those hits now appear beneath the canned answer under a short lead-in, rather than being discarded. The intent still answers first and still counts as an intent match in your diagnostics.

- Only medium-confidence-or-better hits qualify. The bar is deliberately higher than for an ordinary search: an intent has already answered the question, so weak extras beneath it make a confident answer read as hedging.
- Controlled by `intent_supplementary_results`, on by default, and `intent_supplementary_max`, default 3 (§ 17.1).
- If gathering the extras fails for any reason, the canned answer is still returned. The intent response is never put at risk by the addition.

2.18 Also fixed in the 0.9.x series

The credit line in the chat footer was too faint to read. It used the palette's muted UI-furniture colour rather than a text colour, which at 10px failed WCAG AA on every preset — worst on Sunset at 3.39:1, and no better than 3.59:1 on Mocha. It now uses the muted text colour, clearing 4.5:1 on all five presets in both solid and glass modes. No per-theme rule was needed: each preset defines its own value, so the colour resolves dark on the light presets and light on the dark ones by itself.

Phrase detection originally inspected the first and last characters, falling back to `substr()` where `mbstring` is unavailable. `substr()` works in bytes, so the first byte of a curly quote never matched the three-byte character and every smart-quoted phrase fell through to an ordinary search — on precisely the mobile keyboards most likely to produce them. Detection now uses a unicode-safe regular expression, which needs only PCRE and behaves identically with or without `mbstring`.

2.19 What the 0.9.0 documentation audit found

Seven claims in this guide were wrong or incomplete. None was dangerous, but several would have cost somebody an afternoon:

What the guide said	What the code does
The widget is skipped on feeds, embeds, the favicon endpoint and robots.txt	Feeds, embeds and robots.txt. There is no favicon check — the claim was invented. Corrected in § 7.1.
Overlap is capped at a quarter of the chunk size if set too high	The quarter-size fallback applies only when the overlap is greater than or equal to the chunk size. An overlap of 200 on a 250-word chunk is accepted as given. Corrected in § 11.6.
The query log stores query text, intent, score, count, feedback, session and hashed IP	It also stores the normalised token list and the top result's document ID, and truncates the query text to 500 characters. A privacy section should list everything. Corrected in § 14.1.
The /ask endpoint accepts a query	It truncates to 500 characters silently rather than rejecting longer input. Now stated in § 13.1.
Feedback returns 410 when expired	True, but the success shape was never documented. Added to § 13.2.
Two filters and one constant	There is a third filter, <code>mbr_isa_stopwords</code> , which has existed since the tokeniser landed and was never documented. Added to § 17.3.
Every setting listed lives in the settings option	Nine of them are absent until set, with the default applied in code. Explained in § 17.1.

One behaviour was undocumented rather than misdocumented: the shipped intents and synonym groups are used whenever the stored list is empty, not only on a fresh install, so deleting your last intent brings all five

defaults back. That is now in § 8.2.

Why this needed doing. Three of the last four releases fixed something the guide already claimed worked — the glass text colour, the PDF visibility rule, the nonce. In each case the documentation described what was intended and the code did something else, and the gap surfaced as a bug report rather than a review. Checking the claims directly is cheaper than waiting for each one to break.

2.20 Every query failed for logged-out visitors on cached sites

The symptom was that the assistant answered “Something went wrong. Please try again.” to every question — but only for visitors who were not logged in. Testing while logged into WordPress showed nothing wrong, which is why this could sit unnoticed on a live site.

The widget sent an X-WP-Nonce header with every request. WordPress does not treat that header as optional: `rest_cookie_check_errors()` verifies any nonce a REST request supplies, whatever the login state, and rejects the entire request with 403 `rest_cookie_invalid_nonce` if it does not verify.

The nonce is printed into the page by `wp_localize_script()`, so a cached page carries whichever nonce was current when that page was written. Nonces for logged-out visitors expire after 24 hours. Once a cached page outlived that window, every logged-out visitor posted a stale nonce and every query returned 403 — while the administrator, whose pages are never cached, kept getting a fresh one and saw the widget working.

Neither endpoint sends the nonce now. Both are public and unauthenticated by design and perform no capability check (§ 13.4), so it authorised nothing and could only cause this failure.

If you have a custom integration, `window.mbrAisa.nonce` is still localised for backward compatibility, but do not send it. Any code posting to these endpoints with an X-WP-Nonce header will hit the same 403 on a cached site.

The widget now also logs the HTTP status to the browser console when a request fails. Visitors see the same message whatever went wrong, which is deliberate, but a 401 and a 403 need entirely different fixes — see § 15.

2.21 Sunset failed contrast requirements

The Sunset preset's accent was too light to do the two jobs the widget asks of an accent. It is a background beneath - -mbr - isa - base text on the visitor's message bubble and the send button, and it is itself the text colour for every link. The original `#e07856` was too pale for the first and too pale for the second:

Where the accent appears	Before	After
Visitor's message bubble	2.86:1	6.86:1
Send button icon	2.86:1	6.86:1
Link on the assistant's bubble	1.90:1	4.57:1
Link on the panel background	2.86:1	6.86:1

The accent is now #9c3412 and the link-hover colour #6d2a12. All four cases clear WCAG AA. The base, surfaces and foreground colours are untouched, so Sunset keeps its warm peach character — the accent simply reads as a deeper burnt coral than the original.

Sunset only. The other four presets already passed and are unchanged: Mocha 7.79:1, Forest 7.44:1, Ocean 4.51:1, Slate Light 4.50:1. If you have overridden Sunset's accent in your own CSS, your override still wins — check it against the figures above.

2.22 The mousewheel did nothing over the chat log

On a site using Lenis, Locomotive Scroll, GSAP ScrollSmoother or a similar smooth-scroll library, the wheel did nothing over an open chat panel. The log was still scrollable by dragging its scrollbar or with the keyboard, so the widget looked like it was working — which made this an easy fault to miss and an irritating one to hit.

These libraries bind `wheel` on the window, call `preventDefault()`, and drive the page by transform instead of by native scrolling. That disables every nested scroller on the site unless the scroller explicitly opts out. Nothing about the plugin caused it, but the plugin is what has to opt out.

The widget now does two things at start-up:

- Sets the opt-out attributes the common libraries look for (`data-lenis-prevent`, `data-scroll-ignore`) on the message log. They are inert on sites with no such library.
- Stops wheel events propagating out of the log, so a window-level handler never sees them. The wheel is released at the top and bottom of the conversation, so once the log has nowhere further to scroll the page takes over, which is what a visitor expects.

The log also gains `overscroll-behavior: contain`, so a gesture running off the end of the conversation no longer chains into the page behind the panel, and pull-to-refresh does not fire over it on touch devices.

If the wheel is still dead after updating, the library on your site is likely using a capture-phase handler, which the attributes cover but propagation cannot. Check its documentation for the attribute or selector it uses to exclude a nested scroller, and tell me which library it is — support for it can be added.

2.23 The glass effect made your visitors' messages unreadable

With the glassmorphism effect switched on, the visitor's own message bubble was effectively invisible — the text and the bubble behind it were the same brightness.

Outside glass, that bubble is a solid accent background with `--mbr-isa-base` text, a colour chosen to contrast against the accent. The glass rules replaced the background with a translucent tint but left the text colour alone, so it stayed the colour that only made sense against the solid version. On Mocha that put #1e1e2e text on an effective #282938 background — a contrast ratio of 1.14:1. All five presets were affected; the light ones failed the same way inverted.

The text now uses the palette's normal foreground colour, which is what § 10.3 of this guide had described since 0.6.0 without it being true. On Mocha that is 9.92:1. The bubble also takes a mauve tint rather than sharing the assistant's surface tint, so the two speakers stay visually distinct — a distinction the solid accent used to carry on its own.

Nothing to do. 0.8.4 changed one stylesheet. There is no database change, no settings migration, and no reindex. Clear your page cache and CDN so visitors get the new CSS — the stylesheet is versioned by the plugin version, so an unbumped cache is the only thing likely to hide the fix.

2.24 What 0.8.2 got wrong about PDFs

0.8.2 introduced a Which PDFs setting, defaulting to “only PDFs attached to a published page or post”. The word attached was taken literally: the rule required WordPress's attachment relationship, the `post_parent` field on the attachment record.

WordPress only sets `post_parent` when a file is uploaded from inside the post editor. A file added through Media > Add New, brought in by an import, or restored from a backup has no parent — permanently, even after a dozen published pages link to it. On a typical site that excluded most or all PDFs, and nothing on the screen said so.

The rule also required the parent's post type to be one of the indexed types. That broke PDF-only search entirely: unticking every post type is a documented, valid configuration (§ 6.10), but with no post types enabled no parent could ever qualify, so no PDF could ever be indexed.

2.25 The corrected rule

The question the setting is actually trying to answer is did the site owner publish this file. The honest signal for that is whether something published points at it — not which upload screen it happened to come from.

The recommended setting is now “PDFs published somewhere on the site”. A PDF qualifies when a published, unprotected post either owns the file (`post_parent`) or references it in its content or post meta. Post meta is included because page builders keep their content there — Elementor in `_elementor_data`, others similarly — which would otherwise make every PDF on a builder-built site look unreferenced.

The strict `post_parent`-only behaviour remains available as an explicit choice, and “Every PDF in the Media Library” is unchanged. All three are described in § 11.5. From 0.9.7 the reference test additionally requires the linking post to be of a front-end viewable type (§ 2.5).

The privacy intent is unchanged. A PDF that nothing published points at — an old client document, an internal price list, a file linked only from a draft — is still not indexed. Password-protected posts are still excluded. 0.8.3 widened the rule to what 0.8.2 meant to say, not beyond it.

2.26 Scope and visibility are no longer conflated

Whether a post type is searchable is a scope question, set under Content Sources. Whether an attachment is public is a visibility question, answered by whether anything published links to it. 0.8.2 required both of a PDF's parent; 0.8.3 asks only the second. A PDF-only search — every post type unticked, PDF indexing on — works as documented.

2.27 Migration and the eligibility counter

- Sites that took the 0.8.2 default are moved automatically to the corrected setting on the first admin page load after updating. A value you chose deliberately is left alone — if you read the setting and picked the strict option on purpose, it stays.

- Sites with PDF indexing enabled are prompted to reindex. The fix governs what enters the index; it cannot retroactively add the PDFs 0.8.2 refused. An admin notice appears and clears when a full reindex completes.
- Content Sources reports how many PDFs qualify — for example “14 of 21 PDFs in the Media Library” — and warns in red when the answer is none. A setting that silently matches nothing should not need a reindex to discover.
- Filter: `mbr_isa_pdf_scan_postmeta` turns off the post-meta pass on sites that do not need it (§ 17.3).

2.28 What landed in 0.8.2

0.8.2 was a security and privacy release prompted by an external code review. If you are coming from 0.8.1, all of it still applies:

Area	What changed
Password-protected posts	No longer indexed. A protected post keeps the status publish, so gating on status alone wrote its full body to the index and served it as snippets to visitors who never saw the password prompt.
Query log retention	A daily cleanup now runs, with a retention period of 7, 30 or 90 days, or keep indefinitely. New installs default to 30 days; upgrading sites were set to 90. Before this, nothing ever scheduled the cleanup and the table grew without limit.
PDF extraction	Decompression is bounded. A small compressed stream inside an otherwise-legal PDF could previously expand without limit and exhaust the PHP process.
Feedback ratings	A rating now requires a signed token issued with the answer. Query IDs are sequential, so the endpoint could previously be walked to corrupt the statistics.
Rate limiting	Windows are fixed to the clock rather than extended by each request; forwarded headers are consulted when explicitly trusted; an atomic increment is used where a persistent object cache is available.
Session IDs	Generated with <code>random_bytes()</code> rather than <code>md5(uniqid())</code> . The first query of a session is now logged with the correct session ID.
REST documentation	Corrected. The endpoints were described as nonce-protected; the nonce is deliberately not an authentication boundary. See § 13.4.

2.29 Earlier releases in the 0.8 series

- **0.8.8** — regex trigger limits (§ 8.6) and corrected privacy wording (§ 14.4).
- **0.8.7** — the cached-nonce fix described in § 2.20.
- **0.8.6** — the Sunset contrast fix described in § 2.21.
- **0.8.5** — the smooth-scroll fix described in § 2.22.
- **0.8.4** — the glass readability fix described in § 2.23.
- **0.8.3** — the PDF visibility correction described in §§ 2.24–2.27.

- **0.8.1** — WP-CLI commands (§ 12); contents-page detection and demotion; leader-run and running-header cleanup during PDF extraction (§ 11.5); deep links to the matching passage for posts and pages, and page-accurate links for PDFs (§ 11.7).
- **0.8.0** — passage chunking. Long documents are split into overlapping ~250-word chunks, each ranked as its own BM25 unit and collapsed back to one result per document at query time (§ 11.6). Database schema v2.

CHAPTER 3

How it works

When a visitor submits a question, the plugin runs it through a six-stage pipeline. Understanding this pipeline will help you tune the plugin effectively.

3.1 The query pipeline

- **Phrase check.** If the query is wrapped in matching quotes, the inner text becomes an exact-phrase search: the next two stages are skipped entirely and ranking is followed by a literal filter (§ 2.16).
- **Intent match.** The raw query is checked against configured intent triggers. If any trigger matches as a substring (or regex, if prefixed with `re :`), the matching intent's canned response is returned immediately. Disabled intents are ignored. Search still runs where it produces good hits, which are offered beneath the answer (§ 2.17).
- **Tokenisation.** If no intent matched, the query is lowercased, stripped of punctuation, split on whitespace, filtered against an English stopword list, and each surviving word reduced to its stem using a Porter stemmer.
- **Synonym expansion.** Each stemmed token is looked up in the synonym map. Matching terms add their siblings to the token list.
- **Ranking.** The expanded tokens are scored against the inverted index using BM25, with title matches weighted higher than body matches. Each field is normalised against its own length, and each passage chunk is scored independently.
- **Collapse.** Because a long document occupies several chunk rows, the ranked list is collapsed to the single best-scoring chunk per document, so a visitor never sees the same page listed twice.
- **Formatting.** Results are classified by confidence, a matching message is chosen, and a short highlighted snippet is built around the query terms within the winning passage.

3.1a Exact phrase search

Visitors can quote a query to demand an exact run of words. Both of these search for the phrase rather than the words:

```
"opening hours"  
'delivery to the Isle of Man'
```

Quoting also bypasses intents and synonyms, so it is the way to reach the search index when a canned answer keeps getting in the way. Worth mentioning in your widget greeting if your visitors are the sort to use it — nothing advertises the feature otherwise. See § 2.16 for what it can and cannot match.

3.2 Confidence levels

The responder picks one of four confidence levels based on the top result's BM25 score and how much it dominates the second result. Each level changes both the framing message and how many results are returned.

Level	Rule	Results	Typical framing
High	Top score 1.5 or more, and at least 1.5× the runner-up	1	“Here is what I found.”
Medium	Top score 1.0 or more	Up to 3	“These look relevant.”
Low	Top score above 0 but below 1.0	Up to 3	“I’m not sure, but these might help.”
None	No results	0	“I couldn’t find anything.”

3.3 The four database tables

The plugin creates four custom tables, all prefixed with your WordPress table prefix followed by `mbrisa_`:

Table	Purpose
<code>mbrisa_terms</code>	Dictionary of every unique token, with document frequency for BM25 IDF.
<code>mbrisa_documents</code>	One row per passage chunk. A short page is one row; a long PDF is many, distinguished by <code>chunk_index</code> . Each row stores title, URL, token count, content hash, and the passage text used for snippets.
<code>mbrisa_postings</code>	Inverted index — which terms appear in which chunks, with frequency per field (title / content / excerpt).
<code>mbrisa_queries</code>	Log of every query asked through the widget, with intent match, result count, top score, and feedback. Subject to the retention period set under Privacy (§ 14.5).

CHAPTER 4

Installation

4.1 Requirements

- WordPress 5.8 or later
- PHP 7.4 or later
- Standard MySQL / MariaDB — no special extensions required for core search
- PDF indexing uses the standard zlib and mbstring extensions, which ship with virtually every PHP install
- No external services or API keys; no outbound network access is needed to run (only the optional update check reaches out — see § 4.3)

4.2 Installing the plugin

Because this plugin is distributed through Little Web Shack rather than the WordPress.org repository, installation is a manual upload:

- Download the ZIP from littlewebshack.com.
- In your WordPress admin, go to **Plugins** → **Add New**, then click **Upload Plugin**.
- Choose the ZIP file and click **Install Now**, then **Activate Plugin**.
- On activation, the four database tables are created and the default settings are seeded. Existing published content is not indexed automatically at this point — you need to run a full reindex once (see § 5).

Note. The floating chat widget is off by default on a fresh install. This is intentional — the plugin wants you to run a reindex and check the tokeniser before the widget starts talking to real visitors.

4.3 Automatic updates

The plugin keeps itself up to date through the normal WordPress Plugins screen — no WordPress.org listing required. It periodically checks a public GitHub manifest for the latest version, and when you click update it pulls the package from littlewebshack.com, exactly like any other plugin update.

The update check is a server-side version lookup only. It transmits no visitor data, no site content, and nothing that identifies you or your visitors. If you would rather the plugin made no outbound requests of any kind, you can remove the bundled update checker and update manually instead; see § 14.4.

4.4 Upgrading to 0.8.3 — what to expect

There is no database schema change in 0.8.3; the schema remains v4. What changes is a settings migration and, on sites with PDF indexing enabled, a reindex.

What happens automatically

- On the first admin page load after updating, `pdf_visibility` is examined. If it holds the 0.8.2 default of `attached` — or is missing entirely — it is set to `linked`. Any other value is left alone.
- If PDF indexing is switched on, the index is flagged as needing a rebuild and an admin notice appears explaining why. It clears itself when a full reindex completes.

- Nothing is deleted from the index, and nothing is removed from the query log at the moment of upgrade.

What you need to do

If PDF indexing is off, nothing — 0.8.3 changes no behaviour for posts and pages.

If PDF indexing is on, run a full reindex. Until you do, the PDFs 0.8.2 wrongly excluded stay missing from the index and stay unfindable. Then open **Diagnostics** → **Content Sources** and check the eligibility line beneath **Which PDFs** — it tells you how many files now qualify.

```
# The reliable route, especially with a large PDF library:
$ wp mbr-isa reindex

# Or: Diagnostics -> Index Status -> Run Full Reindex
```

Coming from 0.8.1 or earlier? The 0.8.2 migration runs first, and its reindex requirement is the more urgent of the two: password-protected posts and unattached PDFs indexed by 0.8.1 remain searchable through the public endpoint until the index is rebuilt. Both migrations run in sequence on the same admin page load, and one full reindex satisfies both.

OPcache. On a host running OPcache with `validate_timestamps=0` — SiteGround among others — flush OPcache after uploading the plugin files. The migration is triggered by the version constant in the main plugin file, so stale cached bytecode means the migration never runs and neither the corrected setting nor the reindex notice appears.

Note. If you use a page cache or CDN, the widget's assets and any cached pages may need clearing to pick up front-end changes. The index itself is served through a POST endpoint and is never page-cached.

CHAPTER 5

First-time setup

After activation, open **MBR Site Assistant** → **Diagnostics** in your WordPress admin (look for the chat icon in the sidebar). The diagnostic page is the central hub — from there you can reindex, test queries, configure the widget, choose content sources, set privacy options, and reach the three specialised admin pages via quick-access buttons at the top.

The typical first-run sequence is: choose content sources, reindex, test, set retention, configure the widget, pick a theme, enable the widget.

5.1 Choose your content sources

Start here rather than reindexing first — it saves doing the work twice. The **Content Sources** panel controls what gets indexed. Out of the box that is published posts and pages; tick any additional post types you want included. Custom post types appear in the list automatically once registered.

If you also want the contents of your PDFs to be searchable, tick **Index PDF files**, then check the **Which PDFs** setting beneath it. The recommended option indexes any PDF that a published, unprotected page owns or links to. Once PDF indexing is ticked and saved, the panel reports how many files currently qualify. See § 6.10 and § 11.5.

5.2 Run a full reindex

Scroll to the **Index Status** section and click **Run Full Reindex**. This wipes the index tables and rebuilds them from every published item of the enabled post types, plus your PDFs if PDF indexing is switched on. Indexing 100 posts typically takes a few seconds on a modest host.

When it finishes you should see non-zero values for **Documents indexed**, **Chunks**, **Unique terms in dictionary**, and **Total postings**.

5.3 Test with the chat tester

At the bottom of the diagnostic page, the **Chat Tester** runs the full query pipeline exactly as the public widget would. Type a question you expect the plugin to handle — for example, “how do I contact you” (which should match the built-in Contact intent), or a word you know appears on several pages of your site.

Expand the **Raw response payload** details to see the exact JSON the widget will receive. This is invaluable when diagnosing why a query is not producing the result you expected.

5.4 Set a retention period

Under **Privacy & Query Log**, decide how long query history should be kept. The panel shows how many rows are currently stored, the oldest entry, and when the next cleanup is due. Thirty days suits most sites: long enough to spot patterns in what visitors ask, short enough that you are not accumulating a growing pile of free-text input indefinitely. See § 14.5.

5.5 Configure and enable the widget

Scroll to **Widget Settings**. The important fields are:

Setting	Purpose
Floating widget	Tick to show the chat bubble on every page. Leave off if you only want the widget inline via shortcode.
Position	Bottom-right (default) or bottom-left.
Widget title	Header text in the chat panel. Default: "Site Assistant".
Greeting message	First message the assistant shows when the panel opens.
Input placeholder	Ghost text in the input box.

Save your changes. If you ticked **Floating widget**, the bubble will now appear on the front end of your site.

5.6 Pick a theme (optional)

Click the **Appearance** button at the top of the diagnostic page. The default theme is **Mocha** (Catppuccin dark purple). If a different palette suits your site better, choose one of the four other presets — the live preview shows what the widget will look like, and the glassmorphism toggle adds an optional translucent effect. See § 10.

CHAPTER 6

The diagnostic dashboard

The Diagnostics page at **MBR Site Assistant** → **Diagnostics** is the operational hub of the plugin. Three quick-access buttons at the top take you to the specialised admin pages; the body of the page contains everything else.

6.1 Quick-access buttons

- **Manage intents** — opens the intents admin page (§ 8).
- **Manage synonyms** — opens the synonyms admin page (§ 9).
- **Appearance** — opens the theme and effects admin page (§ 10).

6.2 Plugin Status

Quick sanity check: plugin version, database schema version, server PHP version, and the REST endpoint URL the widget will post to. If you are troubleshooting a broken widget, confirm this URL returns a valid response when you POST to it.

6.3 Database Tables

A four-row table showing whether each of the plugin's custom tables exists. Under normal operation all four should show as present. If any table is missing, deactivate and reactivate the plugin to trigger the activation routine again.

6.4 Index Status

Shows document, chunk, term, and posting counts, plus the timestamp of the last full reindex. Also hosts the **Run Full Reindex** button. A reindex is safe to run at any time — it wipes and rebuilds the index tables from scratch. You should run one after:

- Upgrading to 0.9.2 — required to populate the new PDF link offsets (§ 2.15)
- Upgrading to 0.8.3 with PDF indexing enabled — see § 4.4
- Bulk-importing or bulk-editing posts
- Changing the list of enabled post types
- Enabling PDF indexing, or changing the **Which PDFs** setting or the maximum PDF size
- Adding or removing links to PDFs from published pages, if you want the index to catch up immediately (§ 11.5)
- Changing the chunk size or overlap settings (§ 11.6)
- Modifying the stopword list or tokeniser (unlikely in normal use)
- Restoring from a backup

Note. **Documents** counts distinct posts, pages, and PDFs. **Chunks** counts the passage rows those documents occupy — on a site with several long PDFs the chunk figure will be considerably higher, which is expected and healthy.

6.5 Widget Settings

All of the visible widget configuration lives here. See § 5.5 for a field-by-field breakdown and § 7 for more on widget placement.

6.6 Content Sources

Controls what the assistant indexes. See § 6.10 for the full breakdown.

6.7 Privacy & Query Log

Controls whether queries are logged and how long they are kept, and reports the current state of the log. See § 6.11 for the full breakdown and § 14.5 for what is actually stored.

6.8 Feedback & Queries

Seven-day statistics — total queries, thumbs up, thumbs down, how many returned zero results, and how many hit an intent. Below that is a table of the 30 most recent queries. Rows tinted red are zero-result queries or queries that received a thumbs-down. These are the ones worth looking at first, because each one represents either a missing intent, a missing synonym, or a gap in your site's content.

6.9 Testers

The **Tokeniser Tester** shows exactly what the tokeniser does to any text: original, cleaned, split, after stopwords, after stemming. Useful when a word is unexpectedly not matching — tokenise the query and the page title and compare.

The **Search Tester (raw BM25)** runs the search pipeline without intent matching or synonym expansion, with a full trace of which terms were looked up, their scores, matched documents, top score, and average field length per field. The **Chat Tester (full pipeline)** runs exactly what the public widget runs, showing the conversational output with the confidence badge and the raw JSON payload. This is the primary tool for tuning intents and synonyms.

6.10 Content Sources in detail

Control	Purpose
Post types	A checkbox for each registered public post type. Ticked types have their published content indexed. Defaults to Posts and Pages.
Index PDF files	When ticked, PDFs in the Media Library become searchable. Off by default.
Which PDFs	Three options: PDFs published somewhere on the site (recommended), only PDFs attached to a published page or post (strict), or every PDF in the Media Library. See § 11.5 before changing it.
Currently eligible	New in 0.8.3. Shown when PDF indexing is on: how many of the PDFs in your Media Library qualify under the current setting, with a red warning if the answer is none.
Maximum PDF size	PDFs larger than this (in MB, default 20) are skipped, to protect memory on shared hosting. Range 1–100.

After changing anything here, run a full reindex so the change applies to existing content. Once PDF indexing is enabled, PDFs are indexed on upload, re-indexed if the underlying file is replaced, and removed from the index when deleted or when they stop meeting the visibility rule.

Two details about the post type list are worth understanding:

- **Only publicly-viewable post types are listed.** Search results are returned through a public REST endpoint that performs no capability check at query time, so anything indexed is discoverable by any visitor. Private and internal post types are therefore excluded by design — they would also have no reachable permalink for a result to link to.
- **Unregistered types are preserved, not dropped.** If a post type is enabled but not currently registered — typically because the plugin or theme that registered it has been deactivated — it is shown in a separate group marked (*not registered*) and stays ticked. Saving the form while a plugin is temporarily inactive therefore cannot silently remove its content from your configuration. Untick it to remove it deliberately.

Note. Unticking every post type is a valid configuration — for example, a PDF-only search. It is treated as “index no post types”, not as a reset to the defaults, and since 0.8.3 it no longer prevents PDFs from being indexed.

6.11 Privacy & Query Log in detail

The panel opens with a short status table so you can see the state of the log before changing anything:

- **Rows currently stored** — the size of the query log right now.
- **Oldest entry** — the timestamp of the earliest row still held.
- **Next scheduled cleanup** — when the daily retention task next runs. “Not scheduled” here means WP-Cron has not registered the event; loading any admin page re-registers it.

Below that are two controls:

Control	Purpose
Query logging	Whether queries are recorded at all. Switching this off also disables the thumbs up/down rating buttons in the widget, because a rating is recorded against a logged row.
Keep entries for	Retention period: 7, 30 or 90 days, or keep indefinitely. Older rows are deleted by the daily task. Saving the form applies a shortened period immediately rather than waiting for the next run.

Note. “Keep indefinitely” means exactly that — the table will grow for as long as the plugin is active. It is a legitimate choice if you have a reason to hold the history, but it is a choice rather than a default.

CHAPTER 7

The widget: placement and customisation

7.1 Floating bubble mode

With **Floating widget** ticked in settings, a small chat bubble appears on every page of the front end, pinned to the bottom-right or bottom-left. Clicking the bubble opens the chat panel. The widget is automatically skipped on feeds, embeds, and the robots.txt endpoint.

7.2 Inline mode via shortcode

To embed the widget within the flow of a specific page — for example, a dedicated “Ask” page — use the shortcode:

```
[mbr_isa_chat]
```

The shortcode supports four optional attributes that override the global settings for that one embed:

Attribute	Effect
title	Overrides the widget title shown in the panel header.
greeting	Overrides the first assistant message.
placeholder	Overrides the input placeholder text.
height	Sets the panel height — any valid CSS unit, e.g. 500px, 80vh.

Example:

```
[mbr_isa_chat title="Ask us anything" greeting="What are you looking for?" height="600px"]
```

7.3 Running both at once

It is safe to leave the floating widget on and also place a shortcode on a specific page. Each instance gets its own unique ID and operates independently.

The session *ID* is shared through session storage, so queries asked in either instance are tied together in your query log and survive navigation between pages. The *transcript* is not stored: each instance keeps its own messages, and every panel starts empty on a fresh page load. Switching between the two, or reloading, does not carry the visible conversation across.

7.4 Assets and conditional loading

The widget's CSS and JavaScript are only enqueued on pages where they are actually needed: either because the floating widget is enabled site-wide, or because the current page contains the shortcode. Pages that use neither do not load the assets.

7.5 Theming

Every visual aspect of the widget is themeable from the Appearance admin page (§ 10). Both modes — floating and inline — pick up whichever preset and effects you have configured site-wide. There is no shortcode override for theme; it is intentionally a single setting per site.

7.6 Sites with a smooth-scroll library

If your theme uses Lenis, Locomotive Scroll, GSAP ScrollSmoother or anything similar, those libraries take over the wheel for the whole page and disable nested scrollers unless each one opts out. From 0.8.5 the widget opts its message log out automatically and stops wheel events escaping it, so the conversation scrolls normally while the page still takes over at the top and bottom of the log. See § 2.22.

No configuration is needed, and the opt-out attributes are inert on sites with no such library. If the wheel is still dead after updating, the library is probably listening in the capture phase under a name the widget does not yet know; check its documentation for the attribute it uses to exclude a scroller.

CHAPTER 8

Intents — managing through the admin UI

An intent pairs a list of trigger phrases with a canned response. When a query matches any trigger, the intent's response is returned immediately, with good search hits offered beneath it where they exist (§ 2.17). Intents are the single biggest usability win in a site assistant, because they let you give a perfect answer to every common question.

Intents are managed through a dedicated admin page rather than by editing the option in code. Find it at **MBR Site Assistant** → **Intents**, or via the **Manage intents** button on the diagnostic page.

8.1 The intents page layout

- **Test panel** — type a query and see which intent (if any) would match, with the response, label, and confidence.
- **Existing intents** — one card per intent, each containing its own form.
- **Add a new intent** — a blank card at the bottom.
- **Reset to defaults** — replaces all intents with the shipped defaults (with a confirmation).

8.2 Built-in intents

The plugin ships with five default intents, used as a starting point on first install. Once you save any change, the option becomes your own and the defaults stop applying — but you can always reset to them.

One consequence worth knowing. The defaults apply whenever the stored list is empty, not only on first install. Deleting your last remaining intent therefore brings the five defaults back rather than leaving you with none. If you want no intents at all, keep one and tick **Disabled** on it. The same is true of synonym groups.

ID	Example triggers
contact	contact, email address, get in touch, how do I reach
pricing	how much, what do you charge, pricing, your rates, cost of
services	what services, what do you do, what do you offer, can you build
hours	opening hours, what hours, when are you open, business hours
help	need help, can you help, I am stuck, not working

8.3 Editing an intent

Each intent card has six fields:

- **ID** — lowercase, dashes, underscores. Used internally; appears in the diagnostic logs.
- **Label** — human-friendly name shown in test results and diagnostics.
- **Confidence** — a number between 0.0 and 1.0. When more than one intent matches, the higher confidence wins.

- **Triggers** — one per line. Substring match by default; prefix with `re :` for regex.
- **Response** — the message shown to visitors. Basic HTML allowed (links, bold, italic, line breaks, lists). Sanitised on save.
- **Disabled** — when ticked, the intent is kept in the list but does not fire. Useful for A/B testing or temporarily silencing an intent.

Each card has its own **Save changes** button, so an in-progress edit to one intent can never lose another's edits. Each card also has a **Delete intent** button, with confirmation.

8.4 HTML in responses

Responses go through `wp_kses_post ( )` on save, which permits the same set of tags WordPress allows in post content — including links, bold, italic, line breaks, paragraphs, and lists. Anything outside that whitelist is stripped at save time, so the stored response is always safe to render.

On the front end, intent responses are rendered with the HTML preserved. Search responses use a different code path and remain plain text.

8.5 How matching works

- Triggers are matched case-insensitively as substrings of the raw query.
- A trigger prefixed with `re :` is treated as a regex (useful for word-boundary matches, e.g. `re : \bhours?\b`).
- If more than one intent matches, the one with the highest confidence value wins.
- Disabled intents are skipped during matching.
- Empty queries are never matched; neither are intents missing an ID, triggers, or a response.

8.6 Validation rules

- ID must be unique. Saving a new intent with an existing ID is rejected with a clear error.
- ID must consist only of lowercase letters, numbers, dashes, and underscores.
- At least one trigger is required.
- Each `re :` regex trigger must be a valid pattern. Invalid regexes are rejected on save with the offending trigger named.
- **New in 0.8.8.** At most 10 regex triggers per intent, and each pattern at most 250 characters. Plain substring triggers remain unlimited.
- **New in 0.8.8.** Patterns that nest one repetition inside another — `(a+)+`, `(\s*)*`, `(a|a)+` — are rejected. They are valid regular expressions but can take exponentially long to fail on a subject that does not match, and intent matching runs on the public search endpoint, so a visitor would be the one waiting. Ordinary triggers are unaffected.
- The response, after stripping HTML, cannot be empty.

Tip. Use the **Recent queries** list on the diagnostic page to find questions that returned zero results, then add triggers or intents to cover them.

CHAPTER 9

Synonyms — managing through the admin UI

A synonym group is a list of terms that should be considered equivalent for searching. If any term in a group appears in a query, the others are added before ranking. Matching happens on stems, not raw words, so “shops” and “shop” are already equivalent before synonyms kick in.

Synonym groups are managed at **MBR Site Assistant** → **Synonyms**, or via the **Manage synonyms** button on the diagnostic page.

9.1 Built-in groups

Group	Terms
1	wp, wordpress
2	site, website, webpage
3	ecommerce, e-commerce, shop, store, online shop
4	price, pricing, cost, fee, rate, quote
5	contact, email, reach, get in touch
6	help, support, assistance
7	plugin, extension, add-on, addon
8	theme, template, skin
9	build, develop, create, make
10	seo, search engine optimisation, search engine optimization

9.2 Editing a group

Each group card has a single textarea. Enter one term per line — multi-word phrases like “search engine optimisation” are fine on a single line. Both newline-separated and comma-separated input is accepted on save, so pasting from a thesaurus also works.

A group needs at least two unique terms after parsing. Single-term groups do nothing useful, so they are rejected on save with a clear error.

Each card has its own Save and Delete buttons, mirroring the intents page. The header preview shows the first five terms of the group as a code chip — handy when scanning a long list.

9.3 The test expansion panel

Type a query into the test panel and the page shows two rows of token chips:

- **Original tokens** — the words from your query after stemming.
- **Added by synonyms** — the extra tokens that synonym expansion contributed, highlighted in green.

Each chip shows the stem followed by the original word in lighter brackets when the two differ. So `websit` (`website`) reads naturally, while `post` shows as just `post`. Visitors never see the stemmed form anywhere — this is admin-side feedback only.

Note. The Porter stemmer collapses related words to a common root so the index can match them as one. That is why “website” stems to “websit” and “running” stems to “run”. The bracketed original word is purely for readability in the test panel.

9.4 When to add a synonym vs an intent

Add a **synonym** when the missing phrasing is something visitors say but your content uses a different word. “Delivery” and “shipping” might both refer to the same page. Synonyms make the search find what is already on your site.

Add an **intent** when the question has a canned answer that does not correspond to any single page on your site. “What are your opening hours?” might not have a dedicated page — but you still want a good answer.

CHAPTER 10

Appearance — themes and glass effect

The chat-widget colour scheme is configurable at **MBR Site Assistant** → **Appearance**. The page has three areas: a live preview, a colour-preset selector, and an effects toggle.

10.1 The live preview

The preview renders an actual chat-widget structure using the same CSS as the public widget, over an animated gradient background that slowly cycles through a purple-pink-magenta palette. The moving background is intentional — the glass effect only really shows itself when there is something behind the panel for it to blur.

Clicking a preset card or toggling the glass checkbox updates the preview without saving. You can browse presets freely; nothing is committed until you press **Save appearance**. The preview includes a sample link, so you can check link readability per theme before going live.

10.2 The five presets

Preset	Kind	Description
Mocha	Dark	Catppuccin Mocha — the original default. Deep purple panel with mauve and lavender accents. Suits dark and design-led sites.
Slate Light	Light	Clean neutral greys with indigo accent. The corporate-friendly default — works on almost any site.
Ocean	Light	Soft light blues with teal accent. Reads as friendly and calm; good for service businesses, healthcare, education.
Sunset	Light	Warm peach base with a deep burnt-coral accent. Distinctive and warm; suits hospitality, food, lifestyle sites. The accent was darkened in 0.8.6 for contrast (§ 2.21).
Forest	Dark	Deep green forest with sage accent. The dark counterpart to Mocha for sites that want dark but not purple.

Existing installs upgrading from before v0.6.0 see no visual change — Mocha is the default, and the original Catppuccin Mocha CSS is preserved exactly under that preset name.

10.3 Glassmorphism

The **Effects** section has a single toggle. When enabled, the chat panel becomes translucent with a backdrop blur, the bubble button gets a frosted-glass treatment, and the conversation bubbles get a softer glass overlay. Glass works in combination with any of the five presets:

- On dark presets (Mocha, Forest), glass produces a smoked-glass effect — dark and translucent.
- On light presets (Slate Light, Ocean, Sunset), glass produces a frosted-glass effect — bright and translucent.

Under glass the visitor's own message keeps a translucent mauve tint rather than the solid accent it has otherwise, so a question still reads differently from an answer, and its text switches to the palette's normal foreground colour. **Corrected in 0.8.4** — before that release the tint was applied but the text colour was not, leaving the visitor's message unreadable on every preset (§ 2.23).

On contrast: from 0.8.6 all five presets clear WCAG AA in both modes. Without glass, the visitor's message sits on the solid accent — Mocha 7.79:1, Forest 7.44:1, Sunset 6.86:1, Ocean 4.51:1, Slate Light 4.50:1. With glass it sits on a tint beneath the palette's foreground colour, which is more generous still; Mocha reaches 9.92:1. Sunset needed its accent darkening to get there (§ 2.21), and glass was unreadable on every preset before 0.8.4.

Glass relies on the CSS `backdrop-filter` property, which has wide modern browser support. Older browsers fall back to a partially transparent panel without the blur — readable, just less fancy.

Tip. Glass looks best over rich backgrounds. On a white-on-white page the effect is subtle by design — there is nothing visually interesting behind the panel for the blur to act on. On hero images, gradients, or photo-heavy pages, the effect carries.

10.4 Saving and applying

Clicking **Save appearance** writes both `theme_preset` and `theme_glass` into the existing settings option, preserving every other widget setting. The front-end widget picks up the new theme on the next page load — no cache flush required by the plugin, though your own page, CDN, or browser caches may need clearing to reveal the change.

If a preset change appears to do nothing, the two usual causes are a cached page still carrying the old preset class, and a performance plugin's "remove unused CSS" feature stripping the preset blocks it thinks are unused. Both are covered in § 15, including the one-line console check that tells them apart.

CHAPTER 11

Indexing behaviour

11.1 What gets indexed

By default, published posts and pages are indexed. The indexer pulls the title, post content, and excerpt fields, tokenises them, and stores per-field term frequencies so that title matches can be weighted higher than body matches at query time. PDF files in the Media Library can be indexed too (§ 11.5), and long content of any kind is split into passage chunks (§ 11.6).

Content must satisfy both of the following to be indexed:

Condition	What it excludes
Status is publish	Drafts, pending, private, scheduled, and trashed content.
Post password is empty	Password-protected posts. These keep the status publish, so a status-only check let them through — which is how, before 0.8.2, the body of a protected post could be returned as a search snippet to a visitor who never entered the password.

An item is removed from the index the moment it stops meeting either condition — unpublishing a post or adding a password to it both take effect on save, with no reindex needed. Adding a password to a page also re-evaluates any PDFs attached to it, because a PDF's eligibility depends on what published content points at it (§ 11.5).

11.2 Choosing post types

The post types to index are chosen from the Content Sources panel (§ 6.10). Every registered public post type appears as a checkbox, including custom post types registered by your theme or other plugins. Tick the ones you want, save, and run a full reindex.

The selection is stored in the `enabled_post_types` entry of the plugin settings option. If you prefer to set it programmatically — in a deployment script, or from WP-CLI — the option can still be written directly:

```
$s = get_option( 'mbr_isa_settings', [] );
$s['enabled_post_types'] = [ 'post', 'page', 'product' ];
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

11.3 Automatic re-indexing on save

The indexer hooks `save_post`, `deleted_post`, and `trashed_post`. Every time you save or publish a post of an enabled type, the indexer recomputes the content hash; if it has changed, the document's entry is rebuilt. If the post becomes ineligible (§ 11.1), it is removed. Revisions and autosaves are ignored.

When PDF indexing is enabled, attachments are tracked on `add_attachment`, `edit_attachment`, and `delete_attachment` as well. Because a PDF's file can change without its post fields changing, the content hash also factors in the file's size and modification time — so replacing a file re-indexes it.

Note. Because the number of chunks a document produces can change between saves, a changed document has all of its chunk rows deleted and rewritten rather than updated in place. Document frequencies for affected terms are recalculated as part of the same operation, so the index stays consistent.

11.4 Field weighting

Field	Default weight	Settings key
Title	3.0	field_weight_title
Excerpt	1.5	field_weight_excerpt
Content	1.0	field_weight_body

The BM25 parameters `bm25_k1` (default 1.2) and `bm25_b` (default 0.75) are stored in the same option. Since 0.7.1 each field is length-normalised against its own average rather than against the whole document's length. Before that change, a short title match on a long document inherited the length penalty of the entire body, which pushed long documents — large PDFs above all — unfairly down the results.

11.5 PDF indexing

When enabled under **Diagnostics** → **Content Sources**, PDFs in the Media Library are treated as first-class searchable documents. For each PDF the indexer extracts the text layer, tokenises it through the same pipeline used for posts, and stores it in the same index. Search results for a PDF link straight to the file.

Which PDFs get indexed

This is the setting to understand before enabling the feature. Files in the Media Library are reachable at their uploads URL whether or not anything on your site links to them — so indexing decides whether their contents also become findable by search.

Setting	Behaviour
PDFs published somewhere on the site (<i>recommended, default</i>)	A PDF is indexed when a published, unprotected post either owns the file (the WordPress attachment relationship) or references it in its content or post meta. This covers files uploaded through Media > Add New and then linked from a page, which is how most PDFs arrive.
Only PDFs attached to a published page or post (<i>strict</i>)	The 0.8.2 behaviour. Uses the attachment relationship (<code>post_parent</code>) only — which WordPress sets just for files uploaded from inside the post editor. Correct only on sites where every file is uploaded that way.
Every PDF in the Media Library	The pre-0.8.2 behaviour. Every PDF is indexed regardless of what it is attached to. Appropriate only when you are certain the entire library is intended to be public.

How a reference is matched. The test looks for the file's stored relative path — `2026/08/prospectus.pdf` — rather than a full URL, so it survives `http/https` differences, a changed site URL, protocol-relative links and CDN rewriting. Post content is searched first, then post meta, so page builders that store their content in meta — Elementor among them — are handled. The meta pass can be switched off with the `mbr_isa_pdf_scan_postmeta` filter (§ 17.3).

A matching post is a candidate rather than an answer. Each is re-tested against the same visibility rule used everywhere else before its link counts as evidence of publication, so a page an access-control plugin has restricted cannot publish a file on its behalf (§ 2.1).

The parent's post type does not matter. Whether a post type is searchable is a scope question; whether its attachment is public is a visibility question. A PDF linked from a published product page qualifies even if product is not a ticked post type — and a PDF-only search, with every post type unticked, works as documented.

Why the default is what it is. A Media Library accumulates files: documents uploaded to a draft that was never published, an old price list replaced by a newer one, a client document uploaded to share once. None of those are linked from the site, so nobody finds them — until indexing makes their text searchable. The default assumes a file is public only when something published points at it.

One caveat on live updates. When a page is saved, PDFs it owns are re-evaluated immediately. A PDF that merely *links* from a page it does not own is re-evaluated when the PDF itself is saved, or at the next full reindex — so unpublishing the only page linking to a file leaves that file in the index until you reindex. If that matters on your site, reindex after removing a link.

Extraction: what works and what does not

Extraction is entirely local and dependency-free — pure PHP using only the standard zlib extension. It understands the common ways PDF text is stored and compressed (FlateDecode, ASCII85, and ASCIIHex), and reconstructs readable text including word spacing in justified documents and accented or non-Latin characters. Where a PDF also carries Media Library metadata (title, caption, description, alt text), that is folded in as well.

- **Text layer required.** Born-digital PDFs with standard or embedded fonts extract reliably. PDFs built with heavily subsetted or unusual fonts that lack a Unicode mapping may yield partial text.
- **No OCR.** Scanned or image-only PDFs contain no text layer, only a picture of the page. No optical character recognition is performed — that would require external software, contrary to the self-hosted design. Such files fall back to whatever Media Library metadata exists.
- **Encrypted PDFs are skipped.** They are not decrypted or indexed.
- **Size cap.** Files larger than the configured maximum (default 20 MB) are skipped, to protect memory on shared hosting.
- **Decompression is bounded.** Each compressed stream is inflated under a size cap, with a further budget across the whole document. A PDF crafted so that a small stream expands to an enormous one — a decompression bomb — has its oversized streams refused rather than exhausting the PHP process. Genuine content streams are far below the cap and are unaffected.

Two kinds of page furniture are cleaned during extraction so they do not pollute search results: **leader runs** — the rows of dots joining a contents-page heading to its page number — and the page numbers those runs strand. Both are visual devices rather than text; left in place they swallow most of a snippet's window and inflate the passage length that BM25 measures against. **Running headers and footers** are removed too, detected by agreement between pages rather than by pattern, so no configuration is needed.

Note. On a very large PDF library, the first full reindex extracts every file in one operation, which on a constrained host may approach PHP's time or memory limits. The cleanest answer is to run the reindex from the command line, where those limits do not apply — see § 12.

11.6 Passage chunking

Before 0.8.0, a document was one row in the index no matter how long it was. That works well for pages and posts, but poorly for a thirty-page PDF: a single relevant paragraph was diluted across thousands of unrelated words, and the snippet shown to the visitor was always taken from the beginning of the document — rarely the part that actually matched.

The indexer now splits long content into overlapping passage chunks. Each chunk becomes its own row and its own BM25 scoring unit, so a relevant passage competes on its own merits rather than being averaged out across the whole file.

How it works

- Content is split into windows of roughly 250 words, with a 50-word overlap between consecutive windows. The overlap means a sentence that straddles a boundary is present whole in at least one chunk.
- Content short enough to fit comfortably in one window (up to about 375 words) stays a single chunk. Ordinary pages and posts are therefore unaffected.
- The title is indexed with every chunk, so any matching passage still benefits from title relevance. The WordPress excerpt is indexed with the first chunk only.
- A final short remainder is folded into the preceding chunk rather than being emitted as a tiny fragment, which BM25's length normalisation would otherwise over-reward.
- At query time, the ranked list is collapsed to the best-scoring chunk per document, so a visitor never sees the same page or PDF listed twice.
- The winning chunk's text is what the snippet is built from, windowed around the first matching term — so the visitor sees the passage that caused the match.
- Snippets prefer prose to code. Where a term appears both in a code sample and in prose, the prose occurrence is shown. Code is still fully indexed and searchable.

Tuning

The defaults suit most sites and there is deliberately no admin UI for them. If you do need to change them:

```
$s = get_option( 'mbr_isa_settings', [] );
$s['chunk_size_words'] = 250; // default
$s['chunk_overlap_words'] = 50; // default
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

- Smaller chunks (150–200 words) sharpen precision on documents covering many unrelated topics, at the cost of more index rows.
- Larger chunks (350–500 words) keep more context together, which suits documents with long continuous arguments.

- An overlap equal to or larger than the chunk size is rejected and replaced with a quarter of the chunk size. Values below the chunk size are used as given, so an overlap of 200 words on a 250-word chunk is accepted — and will produce a great many heavily-duplicated rows. Keep it well under half.
- A change to either value requires a full reindex to take effect.

Contents pages

A table of contents lists every heading in a document, which makes it the densest concentration of topic vocabulary in the file — and a poor answer to any question, since the visitor gets section titles rather than prose. Passages that look like a contents page are therefore demoted rather than excluded: their score is multiplied by 0.4, so they surface only when nothing better matches.

- Detection is structural: a run of ascending numbered entries carrying very little connective prose.
- Numbered instructions inside a chapter are not affected — they read as prose within each entry, which is what the test measures.
- Data and reference tables are not affected either: decimals such as 1.5 or 0.75 are not counted as entry markers.
- A passage that mixes a short contents list with real prose is left alone.
- The multiplier is tunable via `contents_score_penalty` (default 0.4). Set it to 1.0 to disable.
- **From 0.9.7, a contents passage also never stands in for its own document when another passage in that document matched — so a contents page can no longer hide the chapter it lists (§ 2.11).**

11.7 Deep links to the matching passage

A search result for a post or page links straight to the text that matched, so the visitor arrives at the relevant passage with it highlighted instead of at the top of the page. This uses the Text Fragments standard — the `#:~:text=` directive appended to the result URL. No JavaScript is involved and nothing is sent anywhere: the plugin builds a longer URL from text it already holds.

Browser support

Chrome 80+, Edge 83+, Firefox 131+, Safari 16.1+, Opera 67+ and Samsung Internet 13+ all honour the directive. A browser that does not simply ignores it and loads the page normally, so a deep link can never break a result — the worst case is the behaviour you had before.

How the phrase is chosen

- The anchor starts at the matched word and runs forward, stopping at the end of the sentence or the end of the block element, whichever comes first.
- Browsers will not match a range that spans a block boundary, so a phrase is never allowed to run from a heading into the paragraph beneath it.
- If the first occurrence of a term sits in a short heading, the next occurrence in the prose below is used instead.
- Long runs use the `textStart`, `textEnd` range form.
- If no suitable phrase can be found, no directive is added and the link behaves exactly as before.

PDFs: opening at the right page

PDF viewers do not implement text fragments; they use the long-established PDF open parameters instead. A PDF result links with `#page=N`, opening the file at the page the matching passage falls on. Chrome, Edge, Firefox and Adobe Reader all honour this; a viewer that does not simply opens the file at the start.

To make that possible, PDF text is extracted page by page, with the page sequence read from the document's own page tree rather than assumed from the order objects happen to appear in the file. Because a passage of a few hundred words routinely runs across a page boundary, the link counts the page boundaries falling before the match within the passage, and points at the page the text is actually on.

New in 0.9.2: the link also carries a vertical target where one can be estimated, so the view opens near the passage rather than at the top of the page. The estimate comes from how far through the page's words the passage starts and the page height from its `MediaBox`, biased upward so the passage is never above the landing point. See § 2.15 for the reasoning and its limits, and note that it needs a reindex to take effect.

Note. The offset is an estimate, not a coordinate. On a page whose text is broken up by figures or tables it can be out by a fair margin — always in the direction of landing too high, so the passage is below the fold rather than above it. The snippet in the chat panel tells the visitor what they are looking for.

Limits

- **The phrase must match the rendered page.** The plugin indexes post content. Where a page builder renders from its own stored data rather than from post content, the two can diverge and the phrase may not be found, in which case the browser loads the page at the top as usual.
- Set `deep_link_results` to `false` in the settings option to switch the behaviour off entirely.

CHAPTER 12

Command line (WP-CLI)

If your host provides SSH access with WP-CLI installed — most quality hosts do, including SiteGround, Kinsta, and WP Engine — the plugin exposes two commands under the `mbr-isa` namespace.

The commands are registered only when WP-CLI is running. Ordinary web requests never load the class, so there is no front-end cost to their existence.

12.1 wp mbr-isa reindex

Runs exactly the same full reindex as the button on the Diagnostics page — wiping the index tables and rebuilding them from every eligible item of the enabled post types, plus PDFs if PDF indexing is switched on.

```
$ wp mbr-isa reindex
Running full reindex...
Success: Indexed 32 document(s) as 118 chunk(s) in 12.482s.
Terms: 3038. Postings: 11370.
```

The advantage over the admin button is that CLI processes are not bound by the web server's `max_execution_time` or the PHP-FPM request memory ceiling. On a site with a large PDF library, this is the difference between a reindex that completes and one that dies halfway through with a gateway timeout.

Tip. This is the tidiest way to perform the reindex required after upgrading (§ 4.4), and the natural last step of a deployment script.

12.2 wp mbr-isa status

Prints a summary of the current index without changing anything — useful as a post-deployment health check.

```
$ wp mbr-isa status
Plugin version: 0.8.3
DB schema:      4
Documents:      32
Chunks:         118
Terms:          3038
Postings:       11370
PDF indexing:   on
Last full:      2026-08-09 21:14:22
```

12.3 Multisite and remote use

Both commands accept the standard WP-CLI global flags, so they work with `--url` on multisite and with aliases for remote servers:

```
$ wp mbr-isa reindex --url=https://example.com/site-two
$ wp @production mbr-isa status
```

12.4 Scheduling a periodic reindex

The plugin does not schedule a reindex itself — content is re-indexed as it is saved, so a routine full reindex is not normally required. (It does schedule a daily query-log cleanup; that is separate, and covered in § 14.5.) If you want a periodic reindex anyway, after a nightly content import for example, a cron entry is the simplest approach:

```
# Rebuild the search index at 04:15 every day
15 4 * * * cd /path/to/wordpress && wp mbr-isa reindex --quiet
```

Note. There is deliberately no command to clear the index without rebuilding it. An empty index with the widget live means every visitor gets “I couldn't find anything”, which is never what you want — and a reindex already starts by wiping the tables.

CHAPTER 13

REST API

The chat widget talks to the plugin via two REST endpoints, both under the `mbr-isa/v1` namespace. Both are public and unauthenticated by design, rate-limited per IP hash, and the feedback endpoint additionally requires a signed token.

Note. If you have a security or REST-hardening plugin installed (such as MBR WP Performance), make sure `mbr-isa/v1` is added to its REST allowlist. Otherwise the plugin will block the widget's requests and visitors will see a generic error message. See § 15.

13.1 POST /wp-json/mbr-isa/v1/ask

Accepts a user question and returns the full structured chat response. The query is truncated to 500 characters before processing; longer input is silently cut rather than rejected. Request body:

```
{
  "query": "how do I contact you",
  "session_id": "optional-32-hex-character-session-id"
}
```

Successful response (intent hit):

```
{
  "type": "intent",
  "intent_id": "contact",
  "intent_label": "Contact",
  "message": "You can get in touch via the contact form...",
  "message_html": "You can get in touch via the <a href=...>...",
  "message_format": "html",
  "results": [],
  "suggestions": ["Ask me something else about this site"],
  "session_id": "...",
  "query_id": 1234,
  "feedback_token": "9f2c... (32 hex chars)"
}
```

Intent responses include `message_html` (the sanitised HTML response) and a `message_format` hint of `html`. The plain `message` field is the HTML-stripped version, kept as a safe fallback for any consumer that cannot render HTML. Search responses remain plain text.

Successful response (search hit, medium confidence):

```
{
  "type": "search_results",
  "confidence": "medium",
  "message": "These look relevant:",
  "results": [
    { "title": "...", "url": "...",
      "snippet": "... <mark>match</mark> ...", "score": 2.14 }
  ]
}
```

```

],
"suggestions": [],
"session_id": "...",
"query_id": 1235,
"feedback_token": "..."
}

```

Note. Result objects describe *documents*, not chunks — chunk collapse happens before the response is built, so each result is a distinct page or PDF. The snippet is drawn from whichever passage matched, and for posts and pages the url may carry a `#:~:text=` deep link to that passage (§ 11.7).

`query_id` and `feedback_token` are present only when query logging is enabled, and always together. When logging is off, neither appears and the widget hides its rating buttons.

13.2 POST `/wp-json/mbr-isa/v1/feedback`

Records a thumbs-up (1), thumbs-down (-1), or neutral (0) against a previously-logged query.

```

{
  "query_id": 1234,
  "token": "9f2c...",
  "feedback": 1
}

```

The token is required as of 0.8.2. It is a truncated HMAC over the query ID and its creation time, signed with your site's nonce salt and issued by `/ask` alongside the query ID. Because query IDs are sequential and one is handed out with every answer, existence alone authorises nothing — without the token the endpoint could be walked to file ratings against other visitors' queries and skew your statistics. The token proves the caller is the client that received that particular answer.

A successful call returns `{ "ok": true }` with status 200. Feedback is only accepted within one hour of the original query; older queries return 410 Gone.

13.3 Error responses

Code	Error	Meaning
400	<code>mbr_isa_invalid_query_id</code>	<code>query_id</code> missing or not a positive integer
400	<code>mbr_isa_invalid_feedback</code>	feedback value not in <code>{-1, 0, 1}</code>
403	<code>mbr_isa_invalid_token</code>	Feedback token missing or does not match the query
404	<code>mbr_isa_not_found</code>	<code>query_id</code> does not exist in the log
410	<code>mbr_isa_feedback_expired</code>	More than an hour has passed since the query
429	<code>mbr_isa_rate_limited</code>	Per-IP rate limit exceeded
500	<code>mbr_isa_db_error</code>	Database write failed when storing feedback

13.4 A note on nonces

Earlier versions of this guide, and the code comments, described these endpoints as nonce-protected. That was wrong, and 0.8.2 corrected the description rather than the behaviour — because the behaviour is deliberate.

Up to 0.8.6 the widget sent an X-WP-Nonce header, which WordPress used to establish the current user for a logged-in visitor. It was never treated as an authentication boundary, for two reasons:

- **There is nothing to authenticate.** A site-search assistant has to answer visitors who are not logged in, so both endpoints are open by design. A nonce check would exclude nobody an attacker could not satisfy by loading a page first.
- **Enforcing it would break the widget on cached sites.** The nonce is printed into the page by `wp_localize_script()`, so a page cache serves one visitor's nonce to everybody. A stale or foreign nonce would fail verification on requests that have no business failing.

From 0.8.7 the widget does not send the header at all. Sending it was actively harmful: WordPress verifies any nonce a request supplies regardless of login state and rejects the request with 403 if it fails, so on a page-cached site the stale nonce baked into the cached HTML broke the widget for every logged-out visitor (§ 2.20). Since neither endpoint performs a capability check, nothing was lost by dropping it.

What actually protects these endpoints is per-IP-hash rate limiting in separate buckets per endpoint (§ 14.6), and — for feedback — the signed token described above.

CHAPTER 14

Privacy, data, and rate limiting

14.1 What the plugin stores

- **Post content, as an inverted index.** Stored across the `mbrisa_terms`, `mbrisa_documents`, and `mbrisa_postings` tables. Derived from content you have already published.
- **Extracted PDF text** (when PDF indexing is enabled), from files already in your Media Library, stored in the same index tables.
- **Passage text.** Each chunk row stores the passage it was built from, so snippets can be generated without re-reading the source file.
- **Query log.** Each query submitted through the widget is logged in `mbrisa_queries` with the query text, the normalised token list derived from it, matched intent (if any), the top result's document ID and score, result count, optional feedback value, session ID, created-at timestamp, and a hashed version of the requester's IP address. Query text and tokens are each stored to a maximum of 500 characters.
- **No personal identifiers.** The plugin does not record user accounts, cookies, emails, or raw IPs.

14.2 The query text is free-form input

Worth stating plainly, because it drives the retention setting: the query log stores what visitors typed. Visitors are not always asking tidy keyword questions. Someone may well type “my order 4471 hasn't arrived, I'm at jane@example.com” into a search box, and that string is what gets stored.

Nothing asks them for it and the plugin does not go looking for it, but a log of free-text visitor input will, given enough time, contain some personal data. Two things follow: keep the retention period no longer than you actually need (§ 14.5), and if you publish a privacy policy, mention that site-search queries are recorded.

14.3 IP hashing

Where an IP address is needed — for rate limiting, and for a loose approximation of unique askers in the query log — it is immediately hashed with SHA-256 and your site's WordPress salt. The raw IP is never stored. The hashed value is not reversible without access to your site's salt.

14.4 Outbound traffic

The plugin's description says no external APIs are used for search and visitor queries never leave your server. That is precise rather than coy: the update checker below is an outbound request, and until 0.8.8 the description claimed more than that by saying nothing left your server at all.

On the visitor path, the plugin makes zero outbound HTTP requests. No query — from the widget, the REST API, or anywhere else a visitor can reach — ever leaves your server. This is a core design constraint, not an optional setting. PDF text is extracted locally too, in pure PHP, with no cloud OCR or extraction service. Ranking, chunking, and snippet generation are likewise entirely local.

The one exception is the update checker (§ 4.3), which performs a periodic server-side version check against a public GitHub manifest and downloads the package from `littlewebshack.com` when you choose to update. It transmits no visitor data, no site content, and no identifiers. If you prefer no outbound requests of any kind, remove the bundled update checker and update the plugin manually.

14.5 Query log retention

A daily task deletes rows older than the configured retention period. Set it under **Diagnostics** → **Privacy & Query Log**:

Option	When it suits
7 days	You review the log regularly and only want recent signal. The smallest footprint.
30 days (<i>default for new installs</i>)	Long enough to spot patterns in what visitors ask, short enough not to accumulate indefinitely.
90 days (<i>set on upgrade from pre-0.8.2</i>)	Seasonal sites, or where you want a quarter's worth of history to compare against.
Keep indefinitely	Only when you have a reason to hold the full history. The table will grow without limit.

- Deletion runs in bounded batches against the `created_at` index, so it stays cheap even on a table that has been growing since 0.6.
- Saving the panel applies a shortened period immediately, rather than leaving the excess rows until the next scheduled run.
- Switching query logging off entirely stops new rows being written but does not delete existing ones — set a retention period, or shorten it, if you want the history cleared.
- The schedule is re-registered on any admin page load, so a cleanup event lost to a WP-Cron problem repairs itself.

14.6 Rate limiting

Both REST endpoints are throttled per IP hash, in separate buckets:

Endpoint	Default limit	Configurable via
<code>/mbr-isa/v1/ask</code>	30 requests / minute	<code>rate_limit_per_min</code> in settings
<code>/mbr-isa/v1/feedback</code>	20 requests / minute	Hardcoded for this release

When the limit is hit the endpoint returns 429 Too Many Requests and the widget shows a “you are asking too quickly” message.

Fixed windows

The counter previously extended its own expiry on every request, so the window only reset after a full minute of silence. A visitor querying steadily — one question every few seconds, never exceeding 30 in any given minute — would accumulate a count that never reset and eventually be throttled anyway. Windows are derived from the clock since 0.8.2, so they end when they end regardless of traffic.

Behind a proxy or CDN

The limiter reads `REMOTE_ADDR` by default. Behind a reverse proxy or CDN that is the proxy's address for every visitor, so they all share one bucket and a single busy visitor can throttle the whole site.

Forwarded headers are therefore consulted — but only when you say the site is behind a proxy, because those headers are set by the client and trusting them by default would let an attacker mint unlimited fresh buckets. Opt in with either:

```
// in wp-config.php
define( 'MBR_ISA_TRUST_PROXY', true );

// or via filter, in a theme or plugin
add_filter( 'mbr_isa_trust_proxy', '__return_true' );
```

With that set, the limiter prefers CF-Connecting-IP, then the first entry of X-Forwarded-For, then X-Real-IP, validating each as an IP address before use, and falls back to REMOTE_ADDR. The query log uses the same resolution, so both agree on which address a request came from.

Concurrency

Where a persistent object cache is available (Memcached or Redis), the counter uses an atomic increment. Without one it falls back to transients, which leaves a small race in which simultaneous requests can each read the same count — bounded by a real window, and unlikely to matter on a site without object caching, which is unlikely to be handling that concurrency in the first place.

CHAPTER 15

Troubleshooting

No PDFs are being indexed, or far fewer than expected

- This is the 0.8.2 bug that 0.8.3 fixes. Check **Diagnostics** → **Content Sources** → **Which PDFs**: if it is set to the strict *attached* option, the plugin is requiring the WordPress attachment relationship, which only exists for files uploaded from inside the post editor.
- Switch to **PDFs published somewhere on the site**, save, and read the eligibility line beneath the setting — it reports how many of your PDFs now qualify.
- Then run a full reindex. The setting governs what enters the index; it does not retroactively add files.
- If the count still reads zero of a non-zero total, nothing published on your site links to those files. Either link them from a published page, or choose **Every PDF in the Media Library** if the whole library is meant to be public.

A specific PDF is not showing up in results

- Confirm **Index PDF files** is ticked and that you have run a full reindex since enabling it.
- Check that a *published, unprotected* page actually references the file. A link from a draft, or from a password-protected page, does not count.
- If the link is built by a page builder that stores content somewhere unusual, the post-meta scan may not find it — attaching the file to the page in the editor gives it the ownership route instead.
- Check the PDF is under the configured maximum size (default 20 MB).
- Scanned / image-only PDFs have no text layer — only their Media Library metadata is searchable. Add descriptive metadata to make such files findable.
- Encrypted PDFs are skipped by design.

A PDF stayed in the index after I unpublished the page linking to it

- PDFs a page *owns* are re-evaluated when that page is saved. A PDF that merely *links* from a page is re-evaluated when the PDF itself is saved, or at the next full reindex.
- Run `wp mbr-isa reindex`, or the Diagnostics button, to bring the index back in line.

The reindex notice will not go away

- It clears when a full reindex completes — not when individual posts are saved. Run **Diagnostics** → **Run Full Reindex**, or `wp mbr-isa reindex`.
- If the reindex ran from WP-CLI and the notice persists, reload any admin page; the notice is cleared as part of the reindex, not on a schedule.

The corrected setting never appeared after updating

- The migration is triggered by the plugin version constant. On a host running OPcache with `validate_timestamps=0`, stale bytecode means the old version constant is still in memory and the migration never runs. Flush OPcache and reload an admin page.
- Confirm the version under **Diagnostics** → **Plugin Status** reads 0.8.3.

Content is missing from search after upgrading from 0.8.1

- Expected if the content was password-protected. That is now excluded by design — see § 11.1.
- If a legitimate published page has vanished, check it is not password-protected and that its post type is still ticked under Content Sources.

Feedback buttons have disappeared from the widget

- The rating buttons only render when the response carries both a `query_id` and a `feedback_token`. If query logging is switched off under Privacy, neither is issued and the buttons are hidden by design.
- If logging is on and they are still missing, check the browser console for a JavaScript error, and confirm the widget's JS is the current build — a cached 0.8.1 `chat-widget.js` will not know to send the token. Clear your page cache and CDN.

Feedback submissions return 403

- The token is missing, malformed, or does not match the query being rated. In normal use this means a stale cached copy of the widget JavaScript is posting the old payload shape — clear caches and hard-reload.
- If you are calling the endpoint directly, note that the token must come from the same `/ask` response that produced the query ID. Tokens are not interchangeable between queries.

The mousewheel does nothing over the open chat panel

- Update to 0.8.5 or later and clear your page cache and CDN — a cached `chat-widget.js` will not carry the fix.
- Confirm the log is actually overflowing: compare `scrollHeight` with `clientHeight` on `.mbr-isa-chat__log`. If they match there is nothing to scroll and the wheel is a red herring.
- If it persists, a smooth-scroll library is intercepting the wheel in the capture phase. Check for one with `!!window.lenis || !!window.ScrollSmoother || !!document.querySelector('[data-scroll-container]')` and consult its documentation for the attribute that excludes a nested scroller.

The widget does not appear on the site

- Check that **Floating widget** is enabled under Widget Settings.
- Confirm your active theme calls `wp_footer()`. If it does not, the floating widget cannot render — use the shortcode instead.
- Open your browser's developer console and look for JavaScript errors from `chat-widget.js`.
- Confirm `window.mbrAisa` is defined with a `restUrl`. If not, something has stripped the localized script tag.

Widget shows “Something went wrong” on every query

Open the browser console — from 0.8.7 the widget logs the HTTP status, and 401 and 403 mean different things. Test in a private window: a fault that only affects logged-out visitors is invisible while you are logged in.

- **403** — something rejected the request before it reached the plugin, which returns no 403 from `/ask` at all. On 0.8.6 and earlier this is the stale-nonce fault: update to 0.8.7 and clear your page cache and CDN (§ 2.20). On 0.8.7 it points at a security plugin or a WAF rule.
- **401** — almost always a security or performance plugin disabling the WordPress REST API for logged-out users. The widget runs on the front end where most visitors are not logged in, so a blanket REST blocker catches every request.

- Check the dev tools Network tab and look at the POST to `/wp-json/mbr-isa/v1/ask`; the response body carries a code field naming the cause.
- If your REST-hardening plugin has an `allowed-namespaces` setting, add `mbr-isa/v1` to it. (MBR WP Performance v1.9.3+ has this under Core → Allowed REST Namespaces.)
- If it offers no whitelist, switch its REST mode to “Default (Enabled)”.

Queries return no results even though the content exists

- Run a full reindex. This is the most common cause on a fresh install.
- Check Index Status shows a non-zero document count.
- Use the Tokeniser Tester on both your query and the page title — see which tokens actually come out.
- If the query uses different vocabulary to your content, add a synonym group (§ 9).
- Confirm the post type is ticked under Content Sources, and that the page is published and not password-protected (§ 11.1).

A PDF indexes as empty, or a very large one is skipped

- 0.8.2 capped how far a compressed stream may be inflated. A legitimate document is far below the cap, but a file with an unusually large single content stream can have that stream refused, leaving less text than expected.
- If you have a genuine document affected by this, the text of the affected stream is simply absent; the rest of the file still indexes. Check the Search Tester to see what did make it in.

A search result is full of dots, or shows a list of section titles

- Both symptoms come from a PDF's contents page winning the match. Reindex on 0.8.1 or later: leader dots and stranded page numbers are removed during extraction, and contents-like passages are demoted.
- If a contents page still wins on a particular query, lower `contents_score_penalty` (§ 17.1) below its 0.4 default.

A custom post type is not being indexed

- Confirm it is ticked under Content Sources, then run a full reindex — ticking a type does not retrospectively index existing content on its own.
- If the type is not in the list at all, it is either not registered or not publicly viewable. Types registered with `'public' => false` are excluded by design (§ 6.10).
- If it appears in the (*not registered*) group, whatever registers it is currently inactive; reactivate it and reindex.

A quoted phrase returns nothing, but I can see the words on the page

- The phrase must survive whole inside one passage chunk. A long phrase spanning a chunk boundary can be missed despite the 50-word overlap — try quoting a shorter, distinctive part of it.
- The words must appear in the indexed text. Where a page builder renders from its own stored data rather than post content, what is on screen and what is indexed can differ (§ 11.7).
- Punctuation inside the phrase is matched literally apart from quotes and whitespace, so a hyphen or an ampersand must match what the page actually uses.
- Check the Search Tester with the phrase unquoted to confirm the words are indexed at all.

An intent fires and I do not want the extra results beneath it

- Set `intent_supplementary_results` to false in the settings option for the pre-0.9.1 behaviour, where an intent ended the pipeline (§ 17.1).
- Or lower `intent_supplementary_max` if you want fewer rather than none.
- Only medium-confidence-or-better hits appear, so if you are seeing weak ones, check the Chat Tester to see what the search is actually scoring.

An intent is firing when it should not

- A visitor can bypass intents entirely by quoting their query, which is often the quickest answer for a one-off.
- Triggers are substrings, so a trigger like `help` will match “I need help finding the shop”. Either remove the trigger or tighten it with a regex prefix (`re:\bhelp\b`).
- Use the Chat Tester to see which intent ID is firing.
- Consider ticking **Disabled** on the intent rather than deleting it.

Synonyms admin shows “websit” instead of “website”

- That is the Porter stemmer doing its job. The index uses the stemmed form internally so “websites” matches “website” matches “webpages”. The test panel shows the bracketed original word alongside precisely so it does not read as a typo. Visitors never see stemmed forms.

The widget says “too many requests”

- You are hitting the per-IP rate limit. Wait a minute and try again.
- If the site is behind Cloudflare or another CDN and every visitor is hitting the limit, the limiter is counting them all as one client — set `MBR_ISA_TRUST_PROXY` (§ 14.6).
- If you are testing programmatically, raise `rate_limit_per_min` in the settings option.

Reindex takes a long time or times out

- Run it from the command line instead: `wp mbr-isa reindex` (§ 12). CLI processes are not bound by the web server's execution time or request memory limits.
- If CLI is not available, temporarily raise `max_execution_time` in `php.ini`.
- The reindex is synchronous and batches 50 posts (and 25 PDFs) at a time.
- PDF extraction is heavier than reading post fields — if a large PDF library is the bottleneck, reindex during a quiet period.

The index has far more chunks than documents

- That is expected from 0.8.0 onwards. Long documents occupy several passage rows each (§ 11.6) — a thirty-page PDF might account for forty chunks on its own. The **Documents** figure is the one that corresponds to pages and files on your site.

Buttons or links in the widget take the host theme's colours

- The signature is that a control looks wrong at rest and corrects itself on hover. That is a specificity fight, not a broken stylesheet: the widget's hover rules carry a pseudo-class and outscore the theme, while the resting-state rules did not until 0.9.1.

- Fixed in 0.9.1 by scoping every resting-state rule for links, buttons and inputs one class deeper. If you are on an earlier version, update.
- If it persists on 0.9.1, your theme is winning with an ID selector or `!important`. Inspect the control, find the rule that is winning in devtools, and add a matching override to Additional CSS scoped under `.mbr-isa-chat`.
- `all: unset` in the widget's own CSS does not help here — it clears the browser's default styling, not another stylesheet's rules.

The theme picker has no effect — the widget is always Mocha

The symptom is that every preset renders identically, usually as Mocha, and the glass toggle appears stuck. Two causes, and one check separates them. In a private window, hard-reload and run `document.querySelector('.mbr-isa-chat').className`.

- **The class names still show the old preset.** The floating widget renders into `wp_footer`, so its class list is part of the page your cache stored — changing the setting cannot alter an already-cached page. Purge your page cache and CDN. Note that logged-in requests usually bypass the page cache, so this stays invisible while you test as an administrator.
- **The class names are correct but the styling is not.** Something is interfering with the stylesheet. The usual culprit is a performance plugin's "remove unused CSS" feature: it decides which rules are needed by scanning the rendered HTML, sees one preset class in use, and strips the other four preset blocks as dead weight. The widget then falls back to the base rules, which are Mocha.

To confirm the second, open the served `chat-widget.css` from the Network tab and search it for `--theme-sunset`. If the block is missing, unused-CSS removal has taken it. Add the stylesheet to that feature's exclusion list, or the `mbr-isa-chat--theme-*` and `mbr-isa-chat--glass` classes to its safelist.

Why this file is a poor candidate for unused-CSS removal. The widget's preset class is set server-side from a stored option and can change without a single character of the page's HTML changing. Any tool that infers which CSS is needed by reading one rendered page will therefore infer wrongly the moment you switch preset. The same reasoning applies to the REST namespace allowlist earlier in this chapter: features that harden or slim a site by inspecting one request tend to misjudge a widget configured elsewhere.

Glass effect does not show on the front end

- Confirm the toggle is on under Appearance.
- Glass requires `backdrop-filter` support in the visitor's browser. Very old browsers fall back to a partially transparent panel without blur.
- Glass is most visible over rich backgrounds. On a plain white page it is subtle by design.

A database table is missing after activation

- Deactivate and reactivate the plugin to re-run the activator.
- Check that the WordPress database user has CREATE TABLE permission.

Reindex finishes suspiciously fast and returns no results

- If a reindex that normally takes several seconds completes in well under one, and searches then return nothing, the index tables are empty: the reindex ran but could not write. This is almost always an out-of-date

database schema.

- Check the DB schema version under **Diagnostics** → **Plugin Status** — it should read 4 — then deactivate and reactivate the plugin and reindex.
- From 0.8.1 the reindex notice reports how many items failed to write, rather than reporting success regardless.

CHAPTER 16

Uninstalling and keeping data

Deactivating the plugin leaves all data intact — the tables, options, and widget settings are preserved so you can reactivate later without losing anything. The scheduled query-log cleanup is unregistered on deactivation and re-registered when the plugin is active again.

Deleting the plugin (via Plugins → Delete) by default drops all four custom tables, all plugin options, all cron hooks, and all transients.

16.1 Keeping data across a reinstall

If you plan to delete the plugin and reinstall later, set the `mbr_isa_keep_data_on_uninstall` option to a truthy value before deletion. With that option set, the uninstall routine skips all cleanup and leaves your index, query log, intents, synonyms, and theme settings untouched.

```
update_option( 'mbr_isa_keep_data_on_uninstall', 1 );
```

16.2 Removing the query log without uninstalling

If your aim is to clear visitor query history rather than remove the plugin, you do not need to uninstall. Set the retention period to **7 days** under **Privacy & Query Log** and save — the cleanup runs immediately on save — then set it back to whatever you want going forward. To stop collecting entirely, untick **Query logging**.

CHAPTER 17

Technical reference

17.1 Plugin settings option

All widget, engine, privacy, and theme configuration lives in one option: `mbr_isa_settings`.

Two kinds of key appear below. Most are written into the option when the plugin is activated, so you will see them if you inspect it. The rest — `chunk_size_words`, `chunk_overlap_words`, `contents_score_penalty`, `deep_link_results`, `theme_preset`, `theme_glass`, and the three `widget_*` overrides — are absent until something sets them, and the default in the table is the fallback the code applies when the key is missing. The effect is the same either way; it matters only if you are reading the option and wondering where a key went.

Key	Default	Purpose
<code>enabled_post_types</code>	<code>['post','page']</code>	Which post types are indexed. Set from Content Sources.
<code>bm25_k1</code>	1.2	BM25 term-frequency saturation constant.
<code>bm25_b</code>	0.75	BM25 length-normalisation constant.
<code>field_weight_title</code>	3.0	Score multiplier for title matches.
<code>field_weight_excerpt</code>	1.5	Score multiplier for excerpt matches.
<code>field_weight_body</code>	1.0	Score multiplier for content matches.
<code>chunk_size_words</code>	250	Target passage chunk size in words.
<code>chunk_overlap_words</code>	50	Words shared between consecutive chunks. Capped at a quarter of the chunk size.
<code>deep_link_results</code>	true	Append a text fragment to result URLs so links land on the matching passage.
<code>contents_score_penalty</code>	0.4	Score multiplier applied to passages that look like a contents page. 1.0 disables the demotion.
<code>index_pdfs</code>	false	Index PDF attachments from the Media Library.
<code>pdf_visibility</code>	'linked'	Changed in 0.8.3. 'linked' indexes PDFs a published, unprotected post owns or references. 'attached' uses <code>post_parent</code> only. 'all' indexes every PDF. See § 11.5.
<code>pdf_max_filesize_mb</code>	20	Skip PDFs larger than this many MB.
<code>query_log_retention_days</code>	30	Days to keep query-log rows. One of 7, 30, 90, or 0 for keep indefinitely.
<code>contents_prefer_body</code>	true	New in 0.9.7. Where a document has a matching passage that is not a contents page, use it in preference to a contents passage. Set false for the pre-0.9.7 behaviour, where a contents page could represent its document.

Key	Default	Purpose
pdf_link_zoom	100	New in 0.9.4. Zoom percentage a positioned PDF link opens at. Clamped to 25–400. Only used when pdf_link_position is on.
pdf_link_position	true	New in 0.9.2. Add a vertical target to PDF result links so they open near the passage rather than at the top of the page. Set false to link with the page number alone, which also leaves the reader's zoom untouched.
intent_supplementary_results	true	New in 0.9.1. Show medium-or-better search hits beneath an intent's canned answer. Set false for the pre-0.9.1 behaviour, where an intent ended the pipeline.
intent_supplementary_max	3	New in 0.9.1. Most supplementary results to show beneath an intent.
log_queries	true	Whether to log queries at all.
rate_limit_per_min	30	Max /ask requests per minute, per IP.
widget_enabled	false	Show floating bubble site-wide.
widget_position	'bottom-right'	'bottom-right' or 'bottom-left'.
widget_title	(empty)	Overrides panel title if set.
widget_greeting	(empty)	Overrides greeting message if set.
widget_placeholder	(empty)	Overrides input placeholder if set.
theme_preset	'mocha'	One of: mocha, slate-light, ocean, sunset, forest.
theme_glass	false	Enable glassmorphism effect.

17.2 Other options

Option	Purpose
mbr_isa_version	Installed plugin version string. Drives settings migrations — 0.8.3 uses it to detect an upgrade and to decide whether the 0.8.2 migration also needs to run.
mbr_isa_db_version	Schema version. 5 from v0.9.2 onwards; 4 from v0.8.1.
mbr_isa_intents	Array of intent definitions.
mbr_isa_synonyms	Array of synonym groups.
mbr_isa_index_status	Cached counts shown on the diagnostic page, plus a reindex_required flag set by a migration and cleared by a full reindex.
mbr_isa_keep_data_on_uninstall	If truthy, skip all cleanup on delete.

17.3 Constants and filters

Name	Type	Purpose
MBR_ISA_TRUST_PROXY	Constant	Set true in wp-config.php to let the rate limiter and query log read forwarded-for headers. Off by default. See § 14.6.
mbr_isa_trust_proxy	Filter	Filter equivalent of the constant, for setting the same thing from a plugin or theme.
mbr_isa_pdf_scan_postmeta	Filter	New in 0.8.3. Return false to skip the post-meta pass when deciding whether a PDF is referenced by published content. Post content is still searched. Useful on large sites that do not use a page builder.
mbr_isa_pdf_reference_candidates	Filter	New in 0.9.8. How many candidate referencing posts are tested for public readability when deciding whether a PDF is referenced. Default 25, clamped to 1–500. Raise it where a file is linked from very many places; overrunning the cap leaves the file unindexed rather than indexing it unchecked.
mbr_isa_stopwords	Filter	Filters the English stopwords list loaded from data/stopwords-en.php before tokenisation. Add terms that are noise on your site, or remove ones that carry meaning in your subject area. Changing the list requires a full reindex, since it alters what was stored as well as what is searched.

```
// Skip the post-meta scan (post content is still checked).
add_filter( 'mbr_isa_pdf_scan_postmeta', '__return_false' );
```

17.4 Scheduled events

Hook	Schedule	Purpose
mbr_isa_cleanup_query_log	Daily	Deletes query-log rows older than the retention period, in bounded batches. Registered on <code>init</code> if absent, so a lost event repairs itself. Cleared on deactivation and uninstall.

17.5 Intent data shape

```
[
  'id'      => 'contact',
  'label'   => 'Contact',
  'triggers' => [ 'contact', 'email', 'get in touch' ],
  'response' => 'You can reach me at <a href="...">...</a>.',
  'confidence' => 1.0,
  'disabled' => false,
]
```

17.6 Database schema notes

Schema version 5 is current, introduced in 0.9.2. The documents table carries:

- `chunk_index` — 0 for the first (or only) passage of a document, incrementing for each subsequent passage.

- A unique key on (post_id, chunk_index). The v1→v2 migration drops the old (post_id) key explicitly, because dbDelta adds keys but never removes them.
- A widened passage column so a full chunk can be held for snippet generation.
- is_contents — flags passages detected as a contents page (§ 11.6).
- page_number — records the PDF page a passage begins on (§ 11.7).

Schema migrations are idempotent and run on the first admin page load after updating; no data is deleted.

Settings migrations are separate and keyed on the plugin version rather than the schema version, so 0.8.2 and 0.8.3 migrations can both run in sequence on a site jumping from 0.8.1.

17.7 File layout

```
mbr-intelligent-site-assistant/
|-- mbr-intelligent-site-assistant.php    Bootstrap + update checker
|-- readme.txt
|-- uninstall.php                        Full-cleanup routine
|-- assets/
|   |-- css/chat-widget.css              Styling + presets + glass
|   |-- js/chat-widget.js                Widget controller
|-- data/
|   |-- stopwords-en.php                 English stopwords list
|-- plugin-update-checker/              Self-hosted update library
|-- includes/
|   |-- class-mbr-isa-activator.php       Tables, options, migrations
|   |-- class-mbr-isa-deactivator.php     Deactivation hooks
|   |-- class-mbr-isa-tokeniser.php       Clean / split / stem
|   |-- class-mbr-isa-bm25.php            BM25 scoring
|   |-- class-mbr-isa-chunker.php         Passage chunking
|   |-- class-mbr-isa-pdf-extractor.php   Pure-PHP PDF text extraction
|   |-- class-mbr-isa-indexer.php         Index build, search, visibility
|   |-- class-mbr-isa-synonyms.php        Synonym expansion
|   |-- class-mbr-isa-intents.php         Intent matching
|   |-- class-mbr-isa-responder.php       Confidence, messages, snippets
|   |-- class-mbr-isa-query-handler.php   Pipeline + feedback tokens
|   |-- class-mbr-isa-rate-limiter.php    Per-IP throttling
|   |-- class-mbr-isa-rest.php            REST route registration
|   |-- class-mbr-isa-frontend.php        Shortcode + floating widget
|   |-- class-mbr-isa-cli.php             WP-CLI commands
|   |-- class-mbr-isa-admin-intents.php   Admin UI for intents
|   |-- class-mbr-isa-admin-synonyms.php  Admin UI for synonyms
|   |-- class-mbr-isa-admin-theme.php     Admin UI for appearance
|   |-- class-mbr-isa.php                 Orchestrator + diagnostics
```

17.8 JavaScript integration

The widget exposes a configuration object at `window.mbrAisa`, populated by `wp_localize_script`:

```
window.mbrAisa = {
  restUrl:      'https://example.com/wp-json/mbr-isa/v1/ask',
  feedbackUrl:  'https://example.com/wp-json/mbr-isa/v1/feedback',
  nonce:        '...',
  strings:      { sending: 'Searching...', networkErr: '...', ... }
};
```

For manual re-initialisation — after an AJAX page swap in a headless theme, for example — the widget exposes `window.MbrAisaChat.init()`.

Note. The JavaScript-side identifiers still use the camelCase `mbrAisa` form, retained from an earlier version for backward compatibility with any custom front-end integration code. All PHP-side identifiers are `mbr_isa`.

Integrating directly? If you call the REST endpoints from your own code rather than the bundled widget, note that a feedback submission must include the token returned by `/ask` (§ 13.2). Code written against 0.8.1 will receive a 403.

CHAPTER 18

Support and version information

Item	Value
Plugin version	0.9.8
Author	Robert Palmer
Distribution	littlewebshack.com
Updates	Self-hosted — manifest on GitHub, package on littlewebshack.com
Licence	GPL-2.0-or-later
Text domain	mbr-isa
Database schema	5
Requires	WordPress 5.8+, PHP 7.4+
Tested up to	WordPress 7.0
WP-CLI	wp mbr-isa reindex · wp mbr-isa status

Upgrading from 0.9.7: the short version

- Nothing to do. No schema change, no settings change, no reindex.
- Flush OPcache if your host runs it with `validate_timestamps=0`, or the background purge is never scheduled. The index stays untidy; nothing leaks.
- Clear your page cache and CDN so visitors get the current stylesheet, or the panel gap in § 2.2 will still be there.

Upgrading from 0.9.1: the short version

- Schema v5. Run a full reindex — `wp mbr-isa reindex` — or PDF links keep landing at the top of the page.
- Flush OPcache if your host runs it with `validate_timestamps=0`, or the migration will not run.
- Confirm **Diagnostics** → **Plugin Status** reads schema 5.

Upgrading from 0.8.4

- As above, plus clear your page cache and CDN so visitors get the current widget assets — a cached `chat-widget.js` will not carry the scroll or nonce fixes.

Upgrading from 0.8.3

- Same as above. Both releases are front-end asset changes; one cache clear covers them.

Upgrading from 0.8.2

- Flush OPcache if your host runs it with `validate_timestamps=0`.

- If PDF indexing is off, you are done — nothing else changes.
- If PDF indexing is on: run a full reindex (wp mbr -isa reindex is the reliable route).
- Check **Diagnostics** → **Content Sources** → **Which PDFs** and read the eligibility count beneath it.
- Clear your page cache and CDN so visitors get the current widget assets.

Upgrading from 0.8.1 or earlier

- Both migrations run on the same admin page load; one full reindex satisfies both.
- The 0.8.2 reindex is the urgent one — password-protected posts indexed by 0.8.1 stay searchable until the index is rebuilt.
- Set a query-log retention period under **Privacy & Query Log**.
- If the site sits behind Cloudflare or another CDN, set MBR_ISA_TRUST_PROXY in wp-config.php.

Free and open source. If this plugin has saved you time, a donation at buymeacoffee.com/robertpalmer is very welcome. For bug reports, feature suggestions, or support questions, please get in touch through littlewebshack.com or via the author's contact channels.